CS336 · Spring 2026 · Lecture 04

# 注意力替代方案与混合专家：从状态递推到稀疏计算

## 如何阅读这份自学增强讲义

原始英文幻灯片完整保留；每组原页之后紧接对应中文讲解。书签可跳转到原页与讲解。页码中的“原页”始终指原始 PDF 页码。

中文部分为自学补充；课堂转述、原论文结论与编者推导应按文中标记区分。第三方保存的字幕仅用于检索定位，不等同于教师审校逐字稿。

学习目标：读完后能够用形状与手算说明线性注意力为何能改变乘法顺序，区分未归一化、核归一化和 softmax 注意力；把 Mamba-2 页的门控状态联系到离散状态空间模型与并行扫描；解释 Gated DeltaNet 如何按键方向修改记忆。随后能够从每个 token 的路由过程计算 MoE 的总参数、激活参数和主要算量，理解负载均衡、通信、稳定性以及 DeepSeek 系列的设计联系。

先修：矩阵乘法与外积、链式法则、softmax、Transformer 残差块、MLP、KV cache，以及显存/计算/通信三类成本。阅读顺序严格沿原稿 1–60 页：先减少跨 token 的交互成本，再减少每个 token 访问的参数，最后回到 MLA 与 MTP 的组合。SSM 离散化和扫描只是原页 7 的编者补充，不代表原课件另有一章 S4 或控制理论。

标记约定：原页图表按课件当时的模型与实验条件解读，不作为今天的排行榜；“编者推导/例子”是为自学新增的推演；“课堂补充”带第三方字幕定位，属于核对后的转述而非教师逐字引语。原稿中的疑似笔误保留，在相邻中文说明中纠正。全篇公式约定 q、k、v 为列向量，序列矩阵按行堆叠；y 可按行向量书写，必要时注明转置。

来源索引：slides 为 Spring 2026 官方课件，captions 为公开录像的第三方字幕缓存；linear、mamba、mamba2、gdn 支持注意力与状态模型核对；switch、deepseek-moe、deepseek2、deepseek3、olmoe、stmoe 支持 MoE 公式与实验边界；megablocks、upcycling、nemotron、dsa 为原页对应方法的原始资料。原页内容是覆盖依据，外部资料只用于核实和补足推导。

# Lecture 4

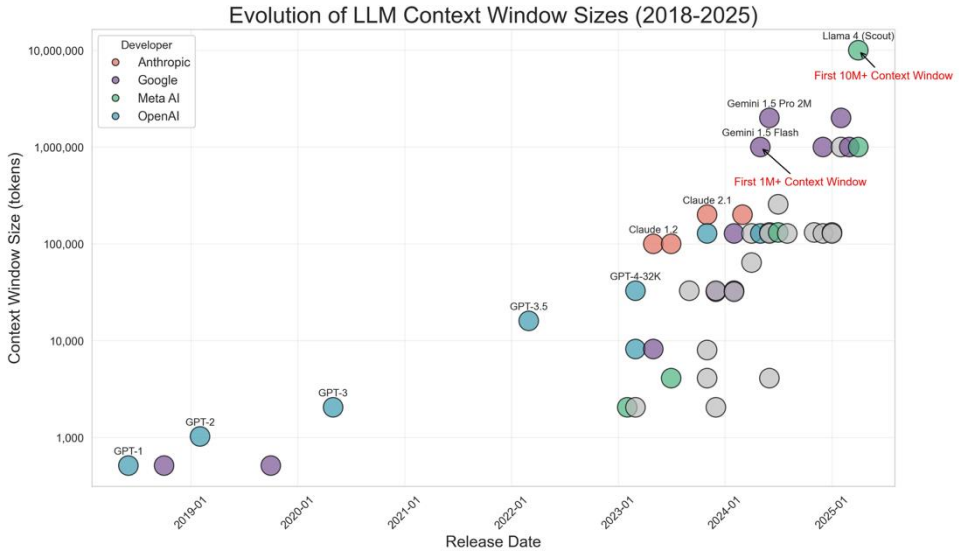
## Attention Alternatives and Mixtures of Experts

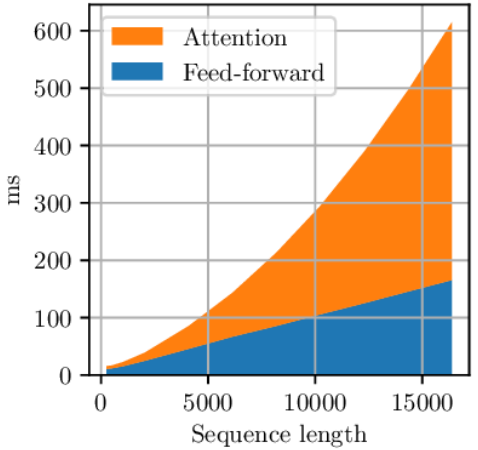
CS336

Tatsu H

# Attention alternatives

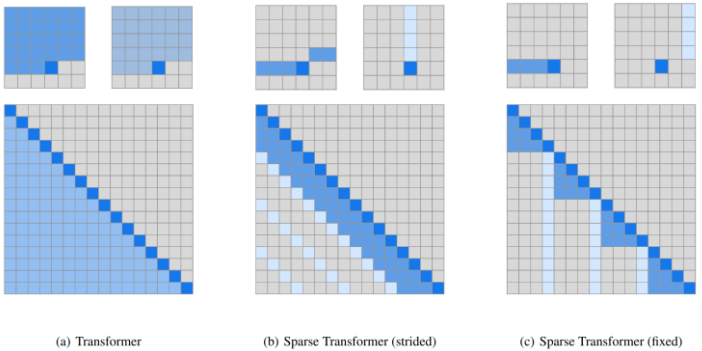
Cost of attention rises with large context sizes… how do we control those costs?



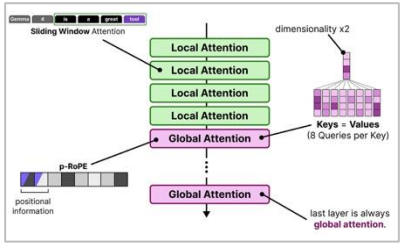




https://www.meibel.ai/post/understanding-the-impact-of-increasing-llm-context-windows

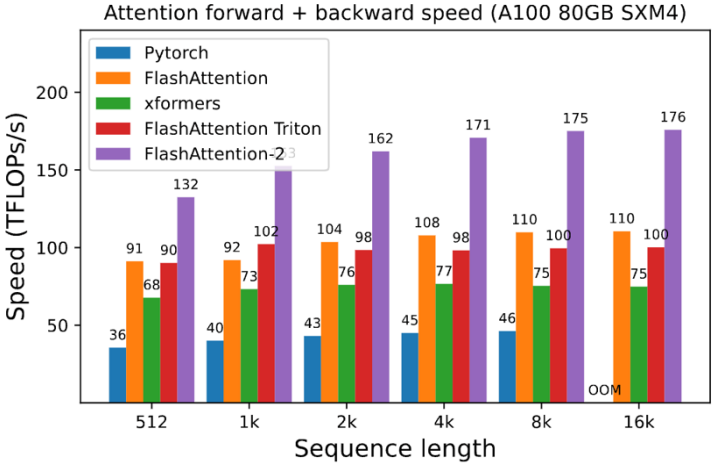
# The ‘basic’ toolkit



(a) Transformer (b) Sparse Transformer (strided) (c) Sparse Transformer (fixed)




Combine local + global attention




Systems engineering

But what if we want more radical and potentially large gains?

第 4 讲 · 原页 1–3 · 中文讲解

# 两种成本：长序列交互与系统执行

## 原页导读与先修联系

原页 2 左图把模型发布时间与可容纳的上下文长度放在一起，纵轴跨度很大；它说明需求的增长，不说明模型能同样可靠地使用每个位置的信息。右图中前馈部分随长度近似线性增加，而注意力部分弯曲上升。设序列长 n、隐藏宽度 d：逐 token 的投影与 MLP 通常是 $O(nd^2)$，全连接注意力打分与聚合是 $O(n^2d)$。长上下文时后者逐渐突出，短上下文时则不一定。

## 编者例子：训练、预填充、解码要分开

固定头数与每头宽度，把长度从 4096 增到 8192，注意力两两交互数量约变成四倍，逐 token MLP 算量约变成两倍。prefill 一次处理整段提示；使用 KV cache 的 decode 每次只新增一个查询，单步读历史 KV 的成本随当前长度增长。不能把全序列 $O(n^2)$ 误解为每个解码步都重算 n×n。KV 缓存本身还随序列长线性增长。

## 读原页 **3** 的两组图

左边的稀疏图中，蓝色位置表示允许的信息路径：局部窗口只连邻近 token，少数全局连接用于跨远距离传递；限制边数会改变模型可见的信息。右边吞吐柱图展示内核工程在具体 GPU 与长度下的效果。FlashAttention 通过分块减少显存读写和中间矩阵存储，仍计算精确 softmax 注意力，不能因此把算术复杂度写成线性。接下来线性注意力改变的是运算形式；比较这三条路时应分别问：删了哪些连接、保留了哪种函数、节省了哪种资源。

本节参考（完整书目见讲义末尾）：[slides]

# Linear attention

Consider the usual attention operation.. $Q \in R^{n \times d_k}, K \in R^{n \times d_k}, V \in R^{n \times d_v}$

$$Attn(Q, K, V) = \rho(QK^\top)V$$

This is quadratic due to $QK^\top$ $(n^2 d_k)$. Can we do better (when $\rho$ is the identity)?

$$(QK^\top)V = Q(K^\top V)$$

This is very silly, but surprisingly important.. We get from $n^2 d_k + n^2 d_v$ to $2nd_v d_k$

[Shen et al 2018, Katharopoulos 2020 for kernel version. Also connections to things like fast weight programmers]

第 4 讲 · 原页 4–4 · 中文讲解

# 结合律如何消除 n×n 中间矩阵

## 编者推导：逐项核对形状和成本

Q、K 的形状是 n×d_k，V 是 n×d_v。先算 $QK^T$ 得 n×n，再乘 V 得 n×d_v；两次矩阵乘法的乘加量级分别为 $n^2d_k$、$n^2d_v$。换顺序后，$K^TV$ 是 d_k×d_v，$Q(K^TV)$ 仍为 n×d_v；两步各需 $nd_kd_v$，合计 $2nd_kd_v$。这里沿用课件的乘加量级记法；若一次乘和一次加各算一个 FLOP，常数还会改变。“线性”指固定特征宽度时随 n 线性，而不是对 d_k、d_v 也线性。

## 编者手算：相同函数的不同计算顺序

取 Q=[[1,0],[0,1]]，K=[[1,2],[3,4]]，V=[[5],[6]]。$K^TV$=[[23],[34]]，所以 $Q(K^TV)$=[[23],[34]]。另一边 $QK^T$=[[1,3],[2,4]]，乘 V 也得 [[23],[34]]。真正的收益在 n 远大于特征宽度时出现；小例子只是验证代数，不是性能基准。

## 成立条件：不能越过 **softmax**

结合律只允许把 $(QK^T)V$ 改成 $Q(K^TV)$。标准注意力是 $\text{softmax}(QK^T/\sqrt{d_k})V$，softmax 作用在每个查询对应的一整行，通常不能拆成 Q 或 K 各自的独立变换后保持精确相等。例如 Q=K=I、V=$[1,0]^T$、忽略缩放时，线性结果是 $[1,0]^T$，softmax 结果约为 $[0.731,0.269]^T$。本页明确假设 ρ 是恒等函数；核特征方法是在选另一种相似度，或近似某个核，不能称为白得的精确 softmax 加速。

本节参考（完整书目见讲义末尾）：[slides] · [linear]

# Recurrent form of linear attention

Recall that in purely linear attention, we consider the reordering

$$(QK^\top)V = Q(K^\top V)$$

This is linear time (great) but the even nicer thing is that this looks like a RNN.

$$S_t = S_{t-1} + k_t v_t^\top \quad \text{and} \quad y_t = q_t^\top S_t$$

This ‘duality’ allows us to train efficiently (using the parallel, quadratic form) and inference efficiently (using the serial, linear form)

(Note: if one weights $S_{t-1}$ by $\gamma$, you get a RetNet)

第 4 讲 · 原页 5–5 · 中文讲解

# 因果状态递推、核特征与归一化

## 编者推导：因果 **mask** 决定状态含义

令 q_t、k_t∈R^{d_k}，v_t∈R^{d_v}，S_0=0。因果输出为 y_t=Σ_{i≤t}(q_t^Tk_i)v_i^T。把 q_t^T 提出来，得 y_t=q_t^TS_t，其中 S_t=Σ_{i≤t}k_i v_i^T，形状 d_k×d_v。因此 S_t=S_{t−1}+k_t v_t^T。外积把一条键值关联写进固定大小矩阵，查询再读取它。若用严格 i<t 的可见性，应先读 S_{t−1} 再更新；本稿沿常见 decoder 的 i≤t 约定，位置 t 的表示用来预测下一 token。

## 双向矩阵式与因果矩阵式的区别

原页沿用 Q(K^TV) 解释双重形式，但整段 K^TV 会包含未来位置。与前缀递推相等的并行形式是 (M ⊙ QK^T)V，其中 M_{ti}=1 当 i≤t，否则为 0，⊙ 表示逐元素乘法。直接给所有 t 使用最终状态 S_n 会泄漏未来。训练时已知所有输入，可以并行投影并用分块或扫描计算各前缀；不必真的串行跑一个普通 RNN，也不必一定显式构造 n×n 矩阵。双重形式提供实现选择，不表示二次算法总是训练最优。

## 编者补充：正核归一化的分母也是状态

若定义 sim(q,k)=φ(q)^Tφ(k)，φ 的维数为 r，采用非负核并希望输出为归一化加权平均，则 S_t=Σ_{i≤t}φ(k_i)v_i^T，z_t=Σ_{i≤t}φ(k_i)，y_t=φ(q_t)^TS_t / [φ(q_t)^Tz_t]。分子是 1×d_v，分母是标量；递推还需 z_t=z_{t−1}+φ(k_t)。分母非零是必要条件；数值实现可以加 ε，但这会略改结果。φ(q)、φ(k) 非负时权重非负；用任意有符号的原始点积，分母可能为零或负，不能直接解释成概率平均。此归一化也不同于层归一化/RMSNorm，更不是把 softmax 精确挪到乘法外。

## 编者手算与记忆容量

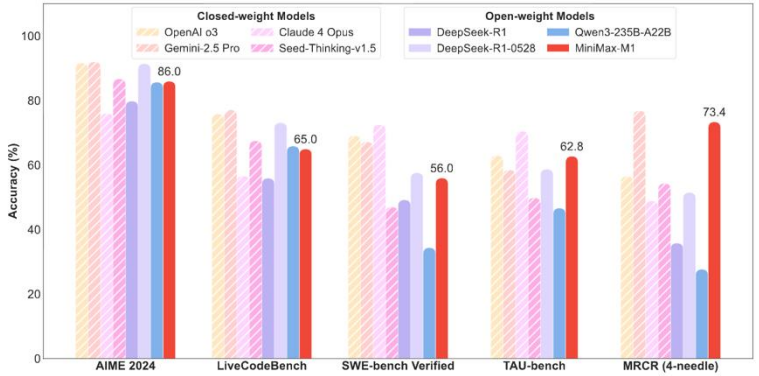
取 φ 为恒等且本例所有向量非负，$k_1=(1,0)^T$、$k_2=(1,1)^T$，$v_1=2$、$v_2=4$，$q_2=(1,1)^T$。$S_1=(2,0)^T$，$S_2=(6,4)^T$，$z_2=(2,1)^T$；未归一化输出为 10，归一化输出为 10/3。直接算两个权重 1、2，也得 (1×2+2×4)/(1+2)。每个位置只保存 r×d_v 的状态和 r 维分母状态，不保存全部 KV；代价是把历史压缩到有限状态，精确检索大量独立关联可能困难。“上下文长度不增加状态大小”不等于“无限记忆”。

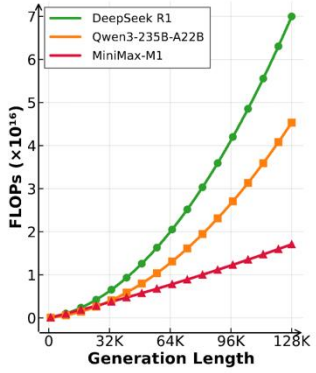
## 课堂补充 · **00:08:04–00:09:34**

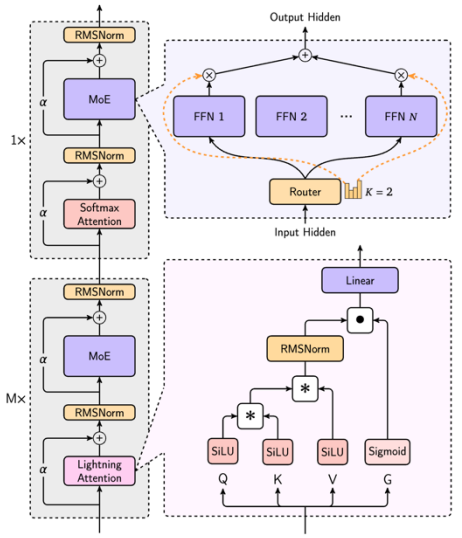
这一段把矩阵乘法重排与 RNN 式推理联系起来，并强调训练和推理可以选择不同计算形式。上述因果 mask、分母递推和数值例子是编者为消除简写歧义而补充的推导，不将它们冒充字幕逐字内容。

本节参考（完整书目见讲义末尾）：[slides] · [linear] · [captions]

# Minimax M1




| Arch. | BBH ↑ | DROP ↑ | MMLU ↑ | CMMLU ↑ | MATH ↑ | GSM8k ↑ | ARC-C ↑ | WG ↑ |
|---|---|---|---|---|---|---|---|---|
| Softmax | 28.2 | 27.4 | 49.3 | **47.3** | 4.6 | **18.8** | 46.4 | 65.6 |
| Hybrid-lightning | **32.2** | **29.0** | **49.5** | 46.0 | **6.8** | 18.5 | **47.4** | **67.5** |

**Minimax M1** (and minimax-text-01) use a 7-to-1 hybrid (7 linear, 1 full) linear attention.

Performance (generally) seems strong, and they show linear scaling in context length

第 4 讲 · 原页 6–6 · 中文讲解

# MiniMax 的混合架构：收益与复杂度边界

## 图表读法

左侧多个任务的柱状图是性能观察，中间曲线比较随生成长度变化的计算成本，右侧结构图显示线性注意力、全注意力与 MoE 等部件。原稿指出 7 层线性配 1 层全注意力。可把它理解为让大部分层压缩历史、少数层仍能直接访问完整上下文，从而在效率与检索能力间折中。底部表格比较特定设置下 softmax 与 hybrid-lightning 的指标，不能推出任意模型改成 7:1 都会提升。

## 编者推导：少量全注意力仍保留二次项

设固定总层数 L，全注意力占比 f=1/8，忽略投影与 MLP 时，prefill 的主要交互成本可写成 $O(L[f\,n^2d+(1-f)nd^2])$。f 很小会显著降低二次项的系数，但只要 f 随 n 保持正数，渐近上仍有 $n^2$ 项。因此原页“linear scaling”应结合所画长度区间和测量指标理解，不能称整个混合网络对任意 n 严格线性。解码缓存也只有线性层变成定长状态，全注意力层仍保留增长的 KV cache。

本节参考（完整书目见讲义末尾）：[slides]

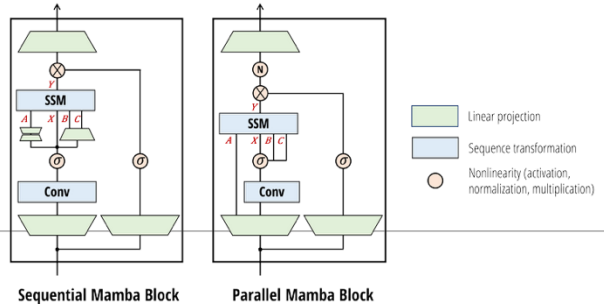
# From linear attention to Mamba-2

Let's generalize linear attention a little bit and add per-position weights..

$$S_t = S_{t-1} + k_t v_t^\top \quad \text{and} \quad y_t = q_t^\top S_t \text{ (Linear attention)}$$
$$S_t = \gamma_t S_{t-1} + k_t v_t^\top \quad \text{and} \quad y_t = q_t^\top S_t + v_t^\top D \text{ with } \gamma_t = f(x_t) \text{ (Mamba-2)}$$

There is a lot more words to justify this (go read the mamba 2 paper) but the mechanics is that we can make linear attention more expressive via gating (gating is good!)

This continues to have duality properties (compute $\gamma$ in parallel, apply duality)



第 4 讲 · 原页 7–7 · 中文讲解

# 从门控记忆到 SSM：离散化与关联扫描

## 原页公式：遗忘、写入与直通

本页改为 $S_t=\gamma_t S_{t-1}+k_t v_t^T$，$y_t=q_t^T S_t+v_t^T D$。$\gamma_t$ 由当前输入计算；$0<\gamma_t<1$ 时衰减旧状态。D 是直通映射，可取 $d_v\times d_v$ 矩阵或逐通道系数。展开得 $S_3=\gamma_3\gamma_2\, k_1v_1^T+\gamma_3k_2v_2^T+k_3v_3^T$，较早信息经过更多门。$\gamma=1$ 退回线性累加，固定 $\gamma$ 给出指数衰减，输入依赖 $\gamma_t$ 则按内容控制保留。此式只刻画 Mamba-2 的递推机制，不含完整工程结构。

## 编者补充：连续模型如何变成离散递推

状态空间模型 SSM 的一个起点是 $dh(\tau)/d\tau=A\,h(\tau)+B\,u(\tau)$，输出 $y(\tau)=C\,h(\tau)+D\,u(\tau)$。$h\in R^m$ 是状态，u 是输入；为简化取 u 为标量，则 A 为 $m\times m$，B 为 $m\times 1$，C 为输出维$\times m$。假设一个长度为 $\Delta$ 的区间内输入恒定，且 A、B 在区间内固定，微分方程的积分解给出 $h_t=A_{bar}\,h_{t-1}+B_{bar}\,u_t$，其中 $A_{bar}=\exp(\Delta A)$，$B_{bar}=[\int_0^\Delta \exp(sA)ds]B$。A 可逆时 $B_{bar}=A^{-1}(\exp(\Delta A)-I)B$；不可逆时积分表达仍然成立，不应硬求 $A^{-1}$。

## 编者手算：离散化不是把导数直接删掉

标量 $A=-1$、$B=1$、$\Delta=\ln 2$，零阶保持离散化得 $A_{bar}=1/2$、$B_{bar}=1/2$。$h_0=0$，输入 $u_1=2$、$u_2=4$，则 $h_1=1$，$h_2=2.5$。这与 $h_t=h_{t-1}+u_t$ 不同。显式 Euler 近似是 $A_{bar}\approx I+\Delta A$、$B_{bar}\approx\Delta B$，只在适当小步长下近似准确，还可能改变稳定性。把离散 $A_{bar}$ 设为 $\gamma_t I$，并把离散输入项组织成 $k_t v_t^T$，就得到原页矩阵状态形式。输入依赖的 $\Delta_t$、$B_t$、$C_t$ 需逐 token 生成；离散系数可吸收进 k、v，勿直接等同连续 B、u。

## 编者推导：为什么能并行扫描

把一步递推写成 $F_t(S)=a_t S+b_t$，$b_t=k_t v_t^T$，$a_t$ 可为标量门。先做第 1 段再做第 2 段的合成是 $(a_2,b_2)\circ(a_1,b_1)=(a_2a_1,a_2b_1+b_2)$。三段展开均为 $a_3a_2a_1\,S+a_3a_2b_1+a_3b_2+b_3$，所以运算满足结合律，但一般不满足交换律。prefix scan 用树形合并求所有前缀合成，可将理想并行深度从 O(n) 降到 O(log n)，同时用工作量高效算法维持 O(n) 次固定维度合成。顺序不能打乱，扫描也没有消除自回归生成对前一个生成 token 的依赖；它加速的是输入已知时的状态计算。

## 编者例子与 Mamba-2 的边界

取 $a_1=a_2=1/2$，$b_1=1$、$b_2=2$，合成映射为 $S\mapsto S/4+2.5$，$S_0=0$ 时得 $S_2=2.5$。稠密 $m\times m$ 转移虽可合成，却可能算得太贵；Mamba-2 利用结构化转移和分块矩阵乘法提速，不能据此声称任意 RNN 都能高效扫描。输入依赖门可预先并行计算，依赖先前状态的非线性门则不满足此前提。
课堂补充 · 00:11:39–00:13:23：本段用位置相关门控解释 Mamba-2 的并行/递推视角；以上积分、Euler 对比和 scan 证明均为编者补充。

本节参考（完整书目见讲义末尾）：[slides] · [mamba] · [mamba2] · [captions]

# Nemotron 3

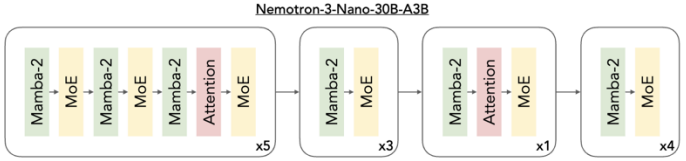



Figure 1 | Nemotron 3 models (e.g., Nemotron Nano 3) leverage a hybrid Mamba-Transformer MoE architecture consisting predominantly of interleaved Mamba-2 and MoE layers, with a select few self attention layers.

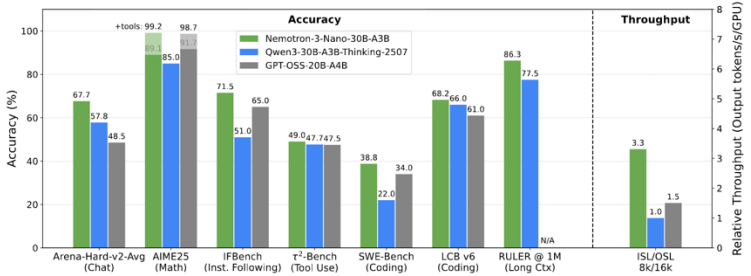



Figure 2 | The hybrid Mamba-Transformer MoE architecture used by Nemotron 3 models can achieve state-of-the-art accuracy on leading reasoning benchmarks and ultra-long-context tasks while providing throughput improvements over similarly sized Transformer MoEs. For details, please see the Nemotron Nano 3 technical report.

Mamba attention hybrid (3-1 ish) – comparable (or better) pref to other similar models

第 4 讲 · 原页 8–8 · 中文讲解

# Nemotron：把状态模型与注意力交错使用

## 结构图与结果图如何连接

原页上方沿层堆叠画出 Mamba、MoE 与 Attention，说明“混合”可以同时发生在序列建模部件和前馈部件。Mamba/注意力决定如何交换 token 间的信息，MoE 决定一个 token 使用哪些前馈参数，它们解决不同维度的计算问题，可以组合。这里的约 3:1 是课件对混合比例的概述，不应把图中所有 MLP/MoE 小块也都算作序列混合层来机械相除。

## 从图中可以与不可以得出的结论

下方柱图同时报告任务质量与吞吐：同类规模的模型可能在某些任务接近、在速度上拉开距离。但吞吐受 batch、上下文长度、硬件及实现影响；激活参数相近也不能消除训练数据与配方差异。作为读者，应看出混合状态模型已经能进入大模型架构，而不应仅凭这组柱图归因于 Mamba 本身。上一节的定长状态节省主要作用在线性/状态层；保留的注意力层仍承担精确访问历史的成本。

本节参考（完整书目见讲义末尾）：[slides]

# Gated delta net (and friends)

Let's generalize things further – gate the input and selectively erase the state.

$$S_t = \gamma_t S_{t-1} + k_t v_t^\top \quad \text{and} \quad y_t = q_t^\top S_t + v_t^\top D \quad \text{with } \gamma_t = f(x_t) \text{ (Mamba-2)}$$

$$S_t = \gamma_t (I - \beta_t k_t k_t^\top) S_{t-1} + \beta_t k_t v_t^\top \quad \text{and}$$
$$y_t = q_t^\top S_t \text{ with } \gamma_t = f(x_t), \beta_t = f(x_t) \text{ (Gated Delta Net)}$$

The gated delta net adds a 'no input operation' gate ($\beta = 0$)

And *erases* anything in the direction of the current key ($I - \beta_t k_t k_t^\top$)

Close relationships to various fast weight programming / test time training ideas.

第 4 讲 · 原页 9–9 · 中文讲解

# Gated DeltaNet：衰减以后按键方向纠错

## 编者推导：把乘法展开成误差更新

原页递推为 $S_t=\gamma_t(I-\beta_t k_t k_t^T)S_{t-1}+\beta_t k_t v_t^T$。设衰减后的状态 $S_{old}=\gamma_t S_{t-1}$，展开得 $S_t=S_{old}+\beta_t k_t(v_t^T-k_t^T S_{old})$。括号是当前 key 从旧状态读出的值与新目标值之间的误差；外积只沿 $k_t$ 方向修改矩阵。这比单纯加 $k_t v_t^T$ 更能处理同一个键的新旧值冲突。这里的 β 控制纠错写入强度，γ 控制全局遗忘，是两个不同的门。

## 编者手算：擦除条件与无输入条件

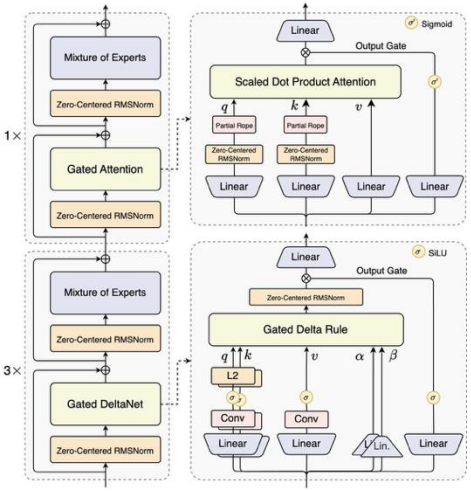
令 $S_{t-1}=[[2,7],[5,9]]$，$k_t=(1,0)^T$，$v_t=(3,4)^T$，$\gamma=1$、$\beta=1$，则 $(I-kk^T)S$ 抹去第一行，外积 $kv^T$ 写入 [3,4]，结果为 [[3,4],[5,9]]。若 $\gamma=1/2$、$\beta=1$，结果为 [[3,4],[2.5,4.5]]：当前键方向被覆盖，正交方向仍保留衰减后的内容。只有 $\|k\|=1$、$\beta=1$ 时 $I-kk^T$ 才是该方向的正交消除；任意未归一化 k 不满足这个结论。Gated DeltaNet 原论文实际对 q、k 做 L2 归一化，课件简式省略了这一条件；因此例子里的单位键也对应到实现中的明确处理。

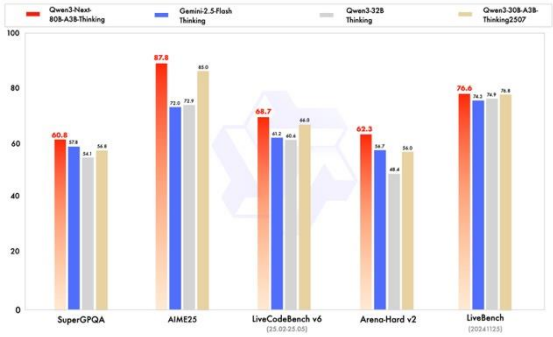
## 三个容易混淆的说法

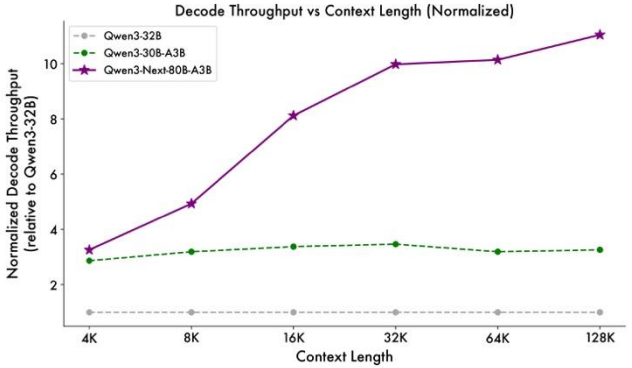
$\beta=0$ 表示没有新的 delta 写入，但状态仍变成 $\gamma S_{t-1}$；除非 $\gamma=1$，否则不是完整的“什么也不做”。β、γ 通常受参数化范围约束，不能将任意数代入都称为稳定门控。把 S 看成临时快权重，可把误差更新联系到对 $1/2\|k^T S-v^T\|^2$ 的一步梯度下降；这是理解 delta 规则的代数视角，不能说推理时一定执行通用反向传播并更新整个模型的慢权重。

本节参考（完整书目见讲义末尾）：[slides] · [gdn]

# Qwen 3.5 / Qwen Next




The newest qwen are 3-1 GDN / Attention hybrids.

Once again, pretty reasonable performance with good inference characteristics

# Hybrid performance

ByteDance | Seed  UC SANTA CRUZ

**A Systematic Analysis of Hybrid Linear Attention**

Dustin Wang*[1], Rui-Jie Zhu*[1,2],
Steven Abreu[3], Yong Shan[2], Taylor Kergan[1], Yuqi Pan[2,4], Yuhong Chou[5], Zheng Li[2],
Ge Zhang[2,6,†], Wenhao Huang[2], Jason Eshraghian[1,†]

[1]UC Santa Cruz, [2]ByteDance Seed, [3]University of Groningen, [4]CASIA, [5]PolyU, [6]M-A-P

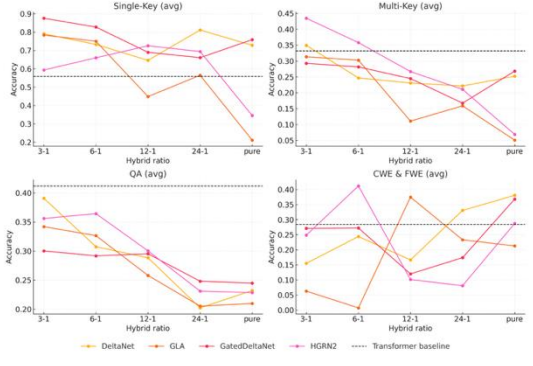
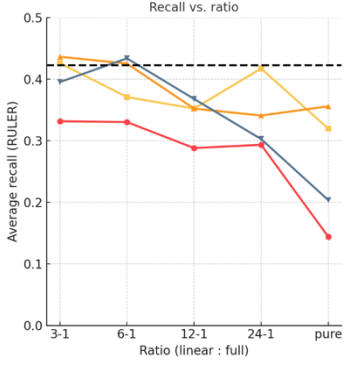



**Figure 4** RULER sub task results based on ratio. RetNet and HGRN model families are omitted as their recall benchmark results were insignificant.

| Model | Update rule ($\mathbf{S}_t$) | Read-out ($o_t$) |
|---|---|---|
| **Vector-valued hidden state (classical / gated RNNs)** | | |
| HGRN[4] | $\boldsymbol{h}_t = \boldsymbol{\alpha}_t \odot \boldsymbol{h}_{t-1} + (1-\boldsymbol{\alpha}_t) \odot \boldsymbol{v}_t$ | $\boldsymbol{o}_t = \boldsymbol{h}_t \odot \boldsymbol{q}_t$ |
| Hawk (RG-LRU)[33] | $\boldsymbol{h}_t = \boldsymbol{r}_t \, \boldsymbol{h}_{t-1} + \boldsymbol{i}_t \odot \boldsymbol{x}_t$ | $\boldsymbol{o}_t = \boldsymbol{h}_t \odot \boldsymbol{q}_t$ |
| **Matrix-valued state via outer products** | | |
| RetNet/Lightning [11, 17] | $\mathbf{S}_t = \gamma \mathbf{S}_{t-1} + \boldsymbol{v}_t \boldsymbol{k}_t^\top$ | $\boldsymbol{o}_t = \mathbf{S}_t \boldsymbol{q}_t$ |
| GLA[34] | $\mathbf{S}_t = \mathbf{S}_{t-1} \odot (\mathbf{1}\boldsymbol{\alpha}_t^\top) + \boldsymbol{v}_t \boldsymbol{k}_t^\top$ | $\boldsymbol{o}_t = \mathbf{S}_t \boldsymbol{q}_t$ |
| Mamba-2[35] | $\mathbf{S}_t = \gamma_t \, \mathbf{S}_{t-1} + \boldsymbol{v}_t \boldsymbol{k}_t^\top$ | $\boldsymbol{o}_t = \mathbf{S}_t \boldsymbol{q}_t$ |
| RWKV-6[36] | $\mathbf{S}_t = \mathbf{S}_{t-1} \mathrm{Diag}(\boldsymbol{\alpha}_t) + \boldsymbol{v}_t \boldsymbol{k}_t^\top$ | $\boldsymbol{o}_t = (\mathbf{S}_{t-1} + (\boldsymbol{d} \odot \boldsymbol{v}_t)\boldsymbol{k}_t^\top)\boldsymbol{q}_t$ |
| HGRN-2/MetaLA[4, 37] | $\mathbf{S}_t = \mathbf{S}_{t-1} \mathrm{Diag}(\boldsymbol{\alpha}_t) + \boldsymbol{v}_t (1-\boldsymbol{\alpha}_t)^\top$ | $\boldsymbol{o}_t = \mathbf{S}_t \boldsymbol{q}_t$ |
| **Delta-rule / controlled-forgetting family** | | |
| DeltaNet[5] | $\mathbf{S}_t = \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top) + \beta_t \boldsymbol{v}_t \boldsymbol{k}_t^\top$ | $\boldsymbol{o}_t = \mathbf{S}_t \boldsymbol{q}_t$ |
| Gated DeltaNet[19] | $\mathbf{S}_t = \alpha_t \, \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top) + \beta_t \boldsymbol{v}_t \boldsymbol{k}_t^\top$ | $\boldsymbol{o}_t = \mathbf{S}_t \boldsymbol{q}_t$ |




Not many controlled ablations, but some evidence of low losses at small hybrid ratios

第 4 讲 · 原页 10–11 · 中文讲解

# 混合比例的实验：表达能力与效率的折中

## Qwen 架构与吞吐曲线

原页 10 的结构图将 Gated DeltaNet 与注意力按约 3:1 交替，旁边同时展示任务柱图和随上下文增长的解码吞吐。读曲线先确认纵轴是归一化 throughput，而非延迟；比值变大说明相对基线优势扩大，不能直接解释为绝对速度随长度提高。状态层降低长上下文中的历史访问量，全注意力层补充直接检索，这是理解结果的合理机制，但图中跨模型结果仍受许多变量影响。

## 原页 11：控制实验比发布榜单更有解释力

左侧四个小图的纵轴都是 Accuracy，分别汇总 Single-Key、Multi-Key、QA、CWE & FWE 子任务；横轴从 3:1、6:1 等混合比例走向纯线性模型。右下图纵轴为 RULER 平均召回率，虚线是 Transformer 基线。随线性层比例增大，多数曲线趋于下降，但并非每个模型和任务都单调，不能从图中读出验证损失值。原页底部虽用 low losses 概括，实际所示图证据是检索准确率与召回率。右上表把向量状态、外积矩阵状态与 delta 规则分组，帮助比较“存什么”和“怎样更新”，不要混淆不同论文的状态转置约定。小混合比例在部分测试中更接近基线，是有条件的实验观察，不是最优比例定理。

## 编者检查问题

若两种混合模型都达到相同验证损失，但一个 KV cache 更小、另一个长距离检索更准，不能只据损失判定全面胜负。应按目标上下文、batch、延迟与检索要求比较。线性层数比例只描述架构，还需知道每层宽度、状态大小、全注意力层位置和内核实现，才能推断实际成本。

本节参考（完整书目见讲义末尾）：[slides]

# Alternative to hybrids: sparse adaptation

## Instead of attending to every token, do sparse attention (DSA)

**Prototype of DSA.** The prototype of DSA primarily consists of two components: a lightning indexer and a fine-grained token selection mechanism.

The **lightning indexer** computes the index score $I_{t,s}$ between the query token $\mathbf{h}_t \in \mathbb{R}^d$ and a preceding token $\mathbf{h}_s \in \mathbb{R}^d$, determining which tokens to be selected by the query token:

$$I_{t,s} = \sum_{j=1}^{H^I} w_{t,j}^I \cdot \mathrm{ReLU}\left(\mathbf{q}_{t,j}^I \cdot \mathbf{k}_s^I\right), \tag{1}$$

where $H^I$ denotes the number of indexer heads; $\mathbf{q}_{t,j}^I \in \mathbb{R}^{d^I}$ and $w_{t,j}^I \in \mathbb{R}$ are derived from the query token $\mathbf{h}_t$; and $\mathbf{k}_s^I \in \mathbb{R}^{d^I}$ is derived from the preceding token $\mathbf{h}_s$. We choose ReLU as the activation function for throughput consideration. Given that the lightning indexer has a small number of heads and can be implemented in FP8, its computational efficiency is remarkable.

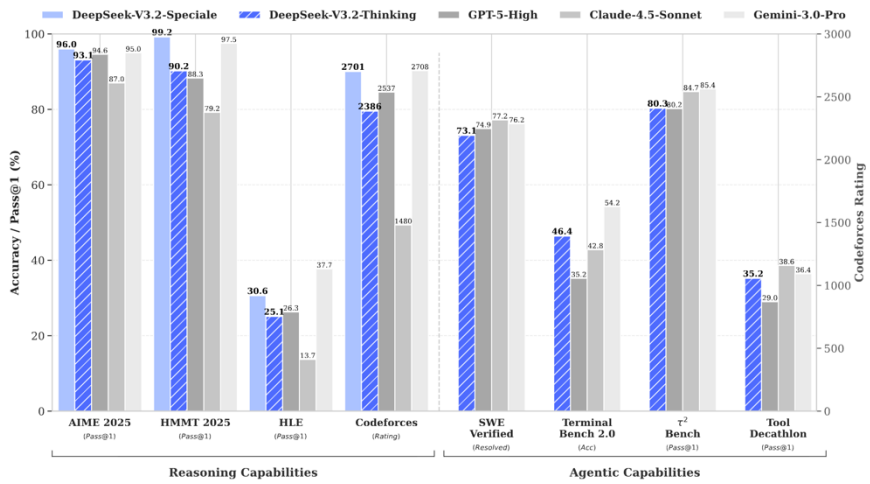
Given the index scores $\{I_{t,s}\}$ for each query token $\mathbf{h}_t$, our **fine-grained token selection mechanism** retrieves only the key-value entries $\{\mathbf{c}_s\}$ corresponding to the top-k index scores. Then, the attention output $\mathbf{u}_t$ is computed by applying the attention mechanism between the query token $\mathbf{h}_t$ and the sparsely selected key-value entries $\{\mathbf{c}_s\}$:

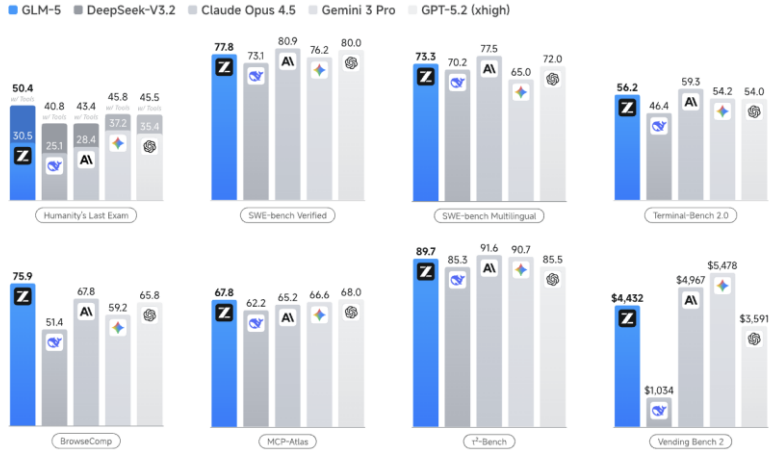
$$\mathbf{u}_t = \mathrm{Attn}\left(\mathbf{h}_t, \left\{\mathbf{c}_s \,\middle|\, I_{t,s} \in \text{Top-k}(I_{t,:})\right\}\right). \tag{2}$$

The indexer can be very lightweight, giving significant gains

Can be ‘post hoc’ adapted after dense short context pretraining

# DSA – Deepseek Sparse Attention (v3.2, GLM5)



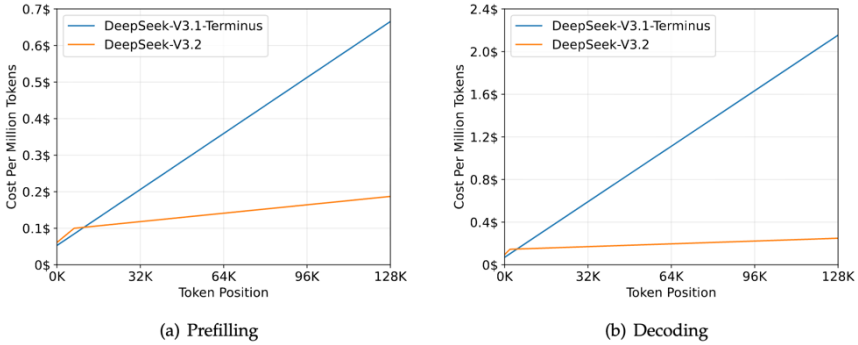




Table 6: RULER benchmark results for the GLM-4.7-Flash with DSA. The warmup-only variant trains only the indexer while keeping the base model frozen, the full DSA variant jointly trains both for 150B tokens.

| | 4K | 8K | 16K | 32K | 64K | 128K |
|---|---|---|---|---|---|---|
| GLM-4.7-Flash | 97.44 | 96.72 | 95.83 | 92.96 | 85.34 | 79.21 |
| GLM-4.7-Flash + DSA warmup | 97.51 | 96.54 | 95.40 | 90.09 | 84.05 | 71.35 |
| GLM-4.7-Flash + DSA | 96.77 | 96.25 | 96.69 | 93.45 | 87.06 | 78.86 |

第 4 讲 · 原页 12–13 · 中文讲解

# DSA：先选历史位置，再做精细注意力

## 原页公式的两阶段含义

DSA 使用轻量 indexer 对当前 query 与历史 token 计算关联分数，选出 top-k 历史位置，再对所选 KV 做真正的注意力。原页 12 的索引式是若干轻量头的 ReLU 点积按查询相关权重相加；它负责排名，不是最终输出的 attention 权重。被选集合只包含可见的历史位置，主注意力再在集合内部归一化并聚合。相较线性注意力把历史混入一个定长状态，DSA 保留可寻址的历史 KV，只稀疏化访问。

## 编者例子：少算主注意力不等于零索引成本

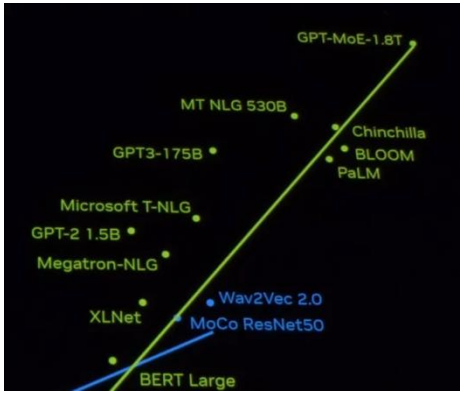
假设当前有 10000 个可见 token，只选 k=512 个，则主注意力每个 query 对应的 KV 聚合项从 10000 缩到 512；但仍需算索引、选择、读取这些 KV。若 indexer 对每个 query 扫描全部过去位置，全序列索引仍可能有二次数量的轻量比较。它通过低维、低精度与高效实现降低常数，不能把整个方法无条件写成严格 O(nk) 或定长缓存。课件说能在稠密短上下文预训练后适配，意味着继续训练索引与稀疏行为，不是推理时随便剪掉低分位置就保证质量不变。

## 原页 **13** 的质量、速度与消融证据

上方是不同模型的任务能力对比，左下分别画 prefill 与 decode 的每百万 token 成本随位置增长，右下是 GLM-4.7-Flash 的 DSA 消融。表中三行是基线、只做 DSA warmup、完整 DSA；没有 MTP。warmup-only 冻结主模型，仅训练 indexer，完整 DSA 则联合训练主模型与 indexer 150B tokens。128K 列依次为 79.21、71.35、78.86，说明单独预热索引器还不足，联合适配可大幅恢复该设置的长上下文表现，但没有在每个长度都严格超过基线。它比跨厂商柱图更能支持适配策略的判断。成本曲线与质量表应一起看，不能只报稀疏化速度而忽略索引器是否选中相关位置。

本节参考（完整书目见讲义末尾）：[slides] · [dsa]

# Mixture of experts




GPT4 (?)

**Mistral AI** @MistralAI

magnet:?xt=urn:btih:9238b09245d0d8cd915be09927769d5f7584c1c9&dn=mixtral-8x22b&tr=udp%3A%2F%2Fopen.demonii.com%3A1337%2Fannounce&tr=http%3A%2F%2Ftracker.opentrackr.org%3A1337%2Fannounce

Grok

deepseek

**DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models**

deepseek

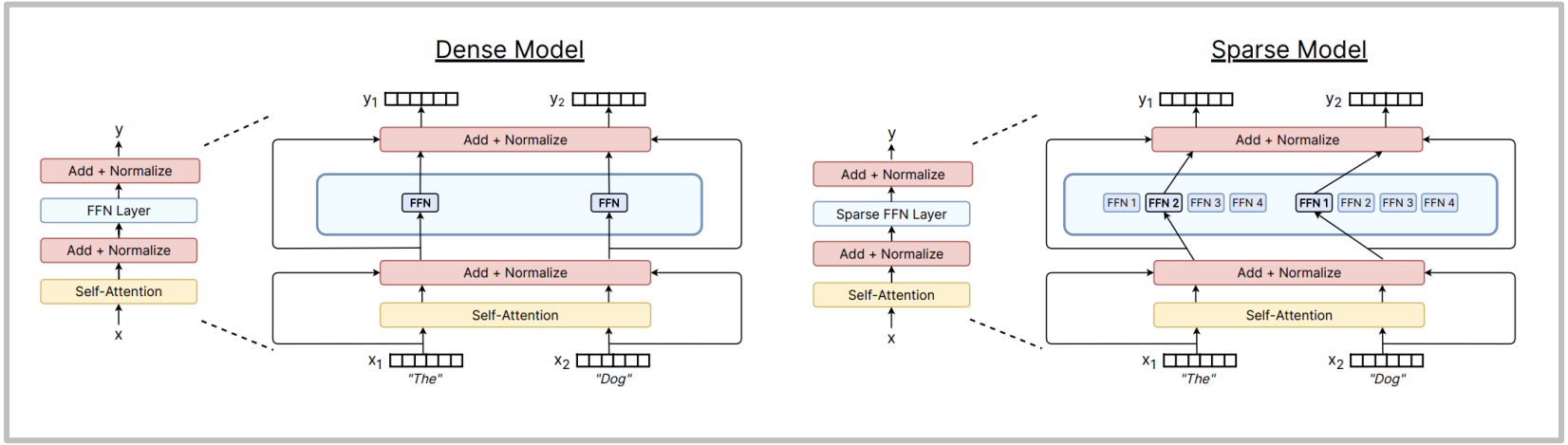
**DeepSeek-V3 Technical Report**




**OLMoE: Open Mixture-of-Experts Language Models**

Niklas Muennighoff[ca] Luca Soldaini[a] Dirk Groeneveld[a] Kyle Lo[a] Jacob Morrison[a]
Sewon Min[a] Weijia Shi[w] Pete Walsh[a] Oyvind Tafjord[a] Nathan Lambert[a]
Yuling Gu[a] Shane Arora[a] Akshita Bhagia[a] Dustin Schwenk[a] David Wadden[a]
Alexander Wettig[ap] Binyuan Hui Tim Dettmers[a] Douwe Kiela[c] Ali Farhadi[aw]
Noah A. Smith[aw] Pang Wei Koh[aw] Amanpreet Singh[c] Hannaneh Hajishirzi[aw]
[a]Allen Institute for AI [c]Contextual AI [w]University of Washington [p]Princeton University
n.muennighoff@gmail.com hannah@allenai.org

# What's a MoE?




[Fedus et al 2022]

Replace big feedforward with (many) big feedforward networks and a selector layer

You can increase the # experts without affecting FLOPs

第 4 讲 · 原页 14–15 · 中文讲解

# MoE 的基本单位：每层、每个 token 选专家

## 从稠密 FFN 到稀疏专家层

Transformer 的普通 FFN 对每个 token 使用同一组权重。MoE 把这一位置替换成 E 个 FFN 及一个 router：对 token 表示 x 算路由分数，选 k 个专家，计算它们的输出，再按门权重加和。专家通常是一层中的子网络，不是若干完整聊天模型开会投票；同一句不同 token、同一 token 在不同层都可能走不同专家。原页 14 的公司和模型拼图是应用背景，其中 GPT4 后带问号，不能把课件猜测改写成已公开确认的结构事实。

## 编者公式与手算：总参数不等于激活参数

设非专家共享部分有 $P_0$ 个参数，每个路由专家有 P_e 个参数，E 个专家中每 token 选 k 个，则总参数约 $P_{total}=P_0+E\,P_e$，每 token 激活参数约 $P_{active}=P_0+kP_e$。若另有 N_s 个始终启用的共享专家，把 $N_sP_e$ 同时加到两式；router 等小项此处略去。取 $P_0=100M$、E=8、$P_e=50M$、k=2，得总参数 500M、激活参数 200M；E 增至 16 而 k 不变，总参数 900M、激活参数仍为 200M。权重必须存储或调入，不能据此只准备 200M 权重的显存就默认能部署全部模型。

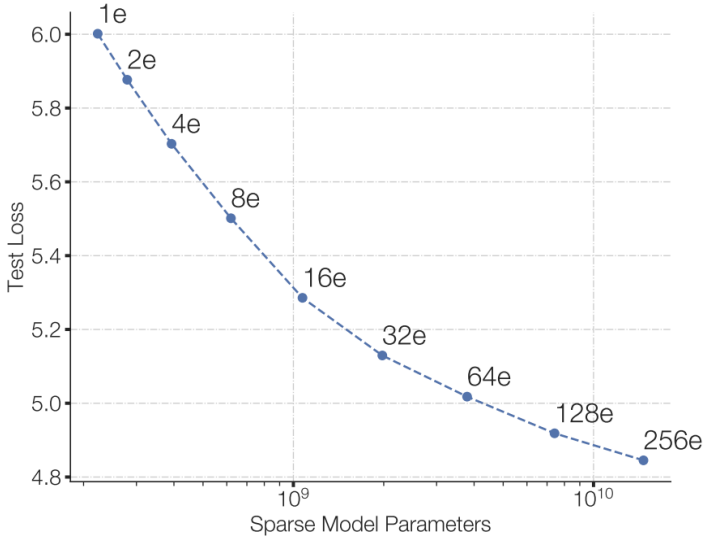
## “不增加 FLOPs”的限定

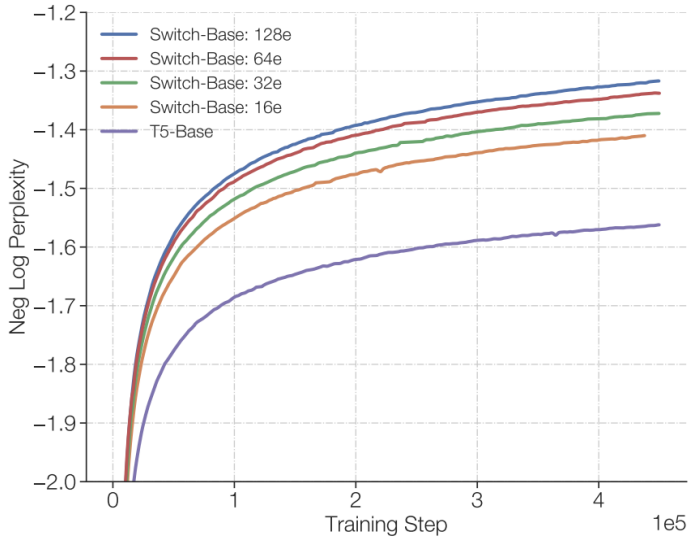
原页的口号主要指固定 k、固定专家宽度时，专家 FFN 每 token 的主算量约正比于 $kP_e$，不随总专家数 E 成比例增长。路由打分约 O(dE)，top-k、dispatch/combine、专家分散产生的通信、负载不均及小矩阵低利用率都不是免费的。训练也只对被选路径计算任务梯度，但一批 token 可能合计访问全部专家；“单 token 稀疏”与“整批只需几名专家”不同。激活参数是计算路径的参数口径，也不是训练中保存的 activation 张量大小。

本节参考（完整书目见讲义末尾）：[slides] · [switch]

# Why are MoEs getting popular?

Same FLOP, more param does better

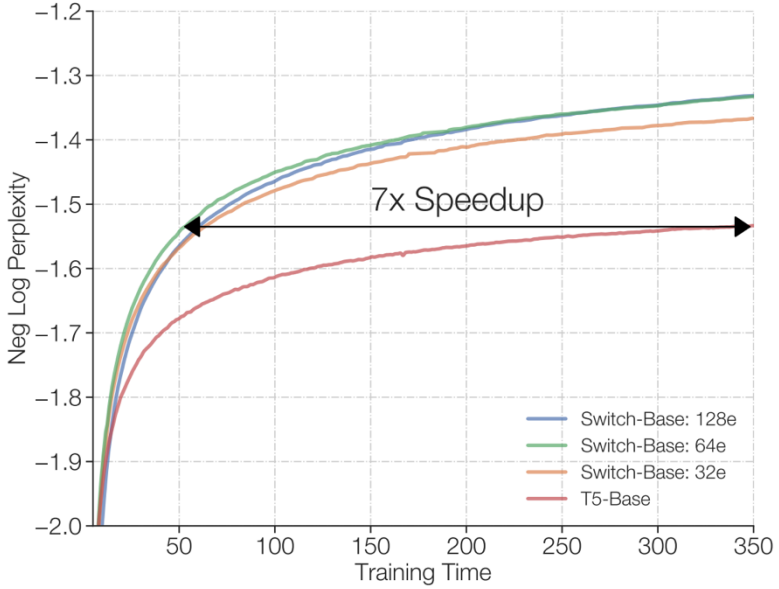



[Fedus et al 2022]

# Why are MoEs getting popular?

Faster to train MoEs



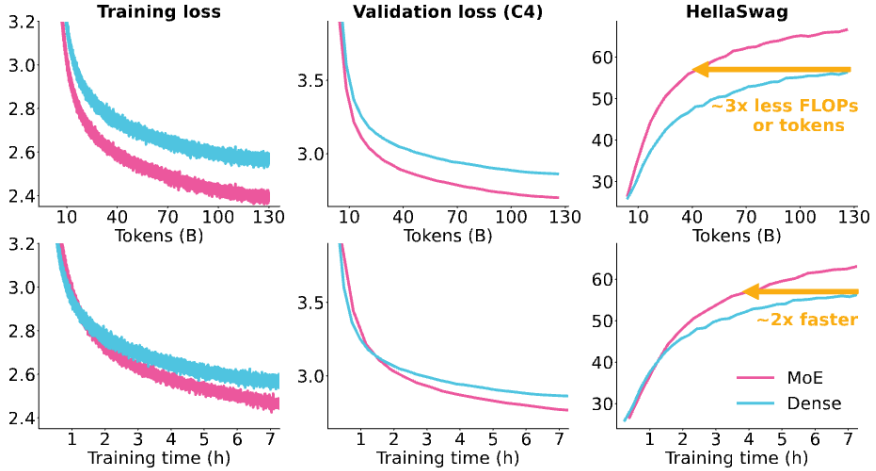
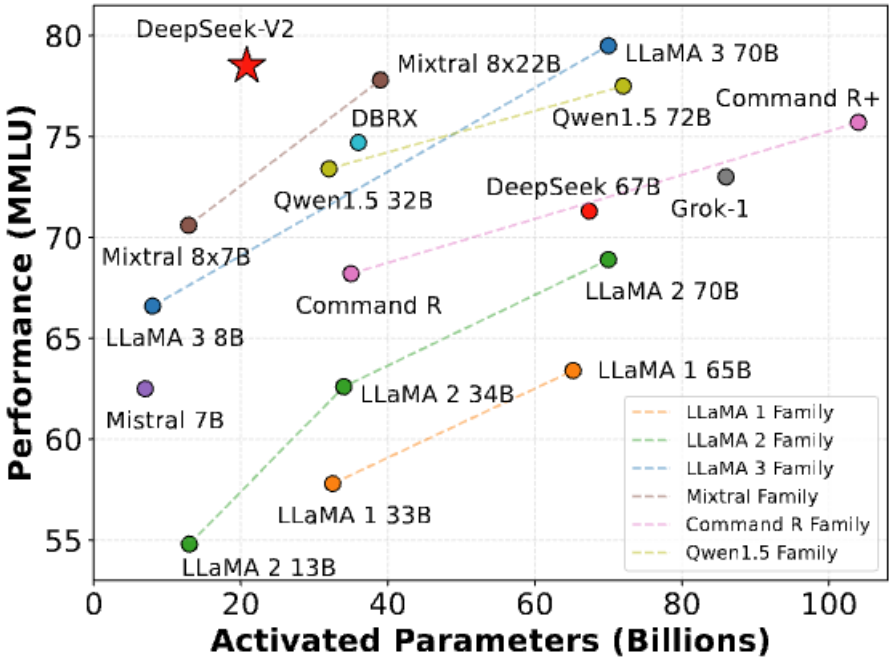




Figure 4: **MoE vs. Dense.** We train a 1.3B parameter dense model and a 1.3B active, 6.9B total parameter MoE model, each on 128 H100 GPUs. Apart from MoE-related changes, we train both

[OlMoE]

# Why are MoEs getting popular?




**Highly competitive vs dense equivalents**

第 4 讲 · 原页 16–18 · 中文讲解

# MoE 为什么有吸引力：容量、算量和墙钟

## 原页 16：同计算路径增加可用容量

左图横轴是稀疏模型总参数，纵轴是测试损失，更多专家的曲线点向下；右图用训练步数观察不同专家数量。它提示在固定每 token 主要算量时，更多可选择的参数可能提高拟合能力。直觉上，不同输入能复用部分参数并访问不同子网络，减轻所有模式竞争同一 FFN 的压力。但专家并非预先赋予“数学/历史”的语义岗位，也不能假设会自动形成清楚的人类类别。

## 原页 17：三种横轴回答三个问题

该页同时出现 token 数、训练时间与指标。对齐 token 数看数据效率，对齐 FLOPs 看计算效率，对齐时间看实际系统速度。图中的 7× 或约 2× 标注是指定硬件、模型、目标质量下的实验结果。若大幅增加总参数而通信很慢，算量优势可能不能兑现为时间优势；反之，专家并行足够好时，较低激活算量可带来明显加速。不要把训练早期斜率直接外推到最终训练预算。

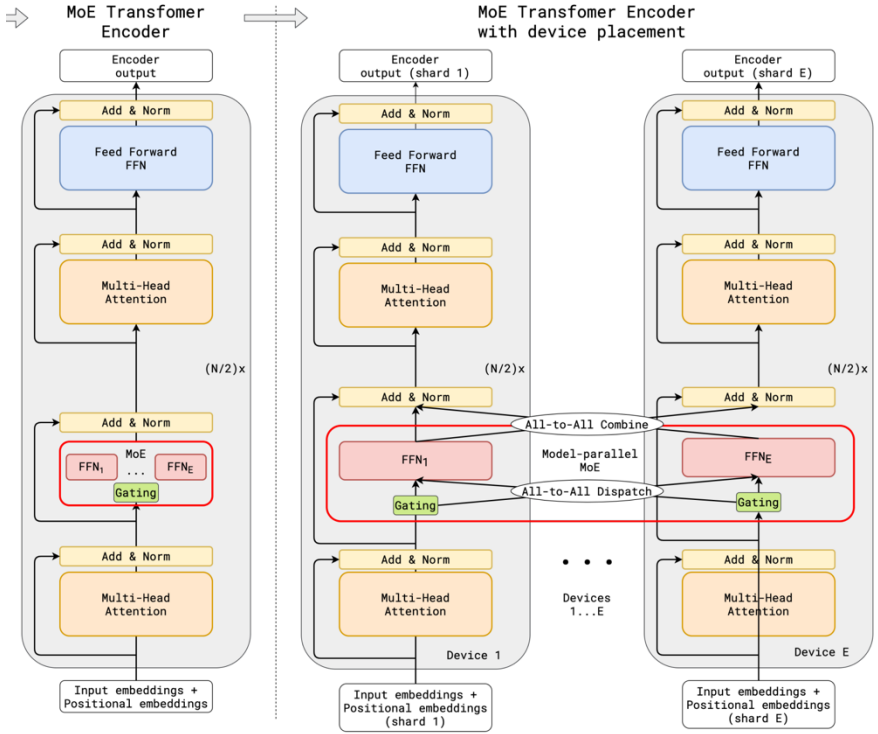
## 原页 18：比较激活参数但保留混杂因素

散点图横轴为 Activated Parameters，纵轴为 MMLU，越靠左上表示在这个计算代理量下更好。DeepSeek-V2 等点说明低激活参数 MoE 能与更大稠密模型竞争，但不是把一个固定模型唯一改成 MoE 的实验：数据、训练 token、评测方式、后训练都有差异。编者建议把“MoE 容量扩大”视为待验证机制，把受控同预算消融视为较强证据，把产品间榜单视为存在性与可行性证据。

本节参考（完整书目见讲义末尾）：[slides]

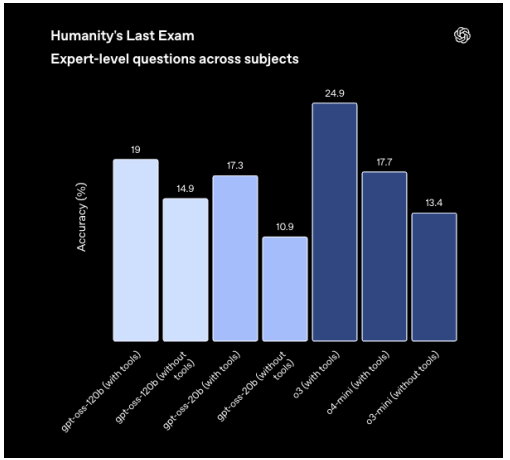
# Why are MoEs getting popular?

Parallelizable to many devices



# Some MoE results – from the west

| Category<br>Benchmark | Llama 4 Maverick | Gemini 2.0 Flash | DeepSeek v3.1 | GPT-4o |
|---|---|---|---|---|
| Inference Cost<br>Cost per 1M input & output tokens (3:1 blended) | $0.19-$0.49[5] | **$0.17** | $0.48 | $4.38 |
| Image Reasoning<br>MMMU | **73.4** | 71.7 | No multimodal support | 69.1 |
| MathVista | **73.7** | 73.1 | | 63.8 |
| Image Understanding<br>ChartQA | **90.0** | 88.3 | | 85.7 |
| DocVQA (test) | **94.4** | — | | 92.8 |
| Coding<br>LiveCodeBench (10/01/2024-02/01/2025) | 43.4 | 34.5 | **45.8/49.2**[3] | 32.3[3] |
| Reasoning & Knowledge<br>MMLU Pro | 80.5 | 77.6 | **81.2** | — |
| GPQA Diamond | **69.8** | 60.1 | 68.4 | 53.6 |




MoEs are most of the highest-performance open models, and are quite quick

# Earlier MoE results from Chinese groups – Qwen

Chinese LLM companies are also doing quite a bit of MoE work on the smaller end

| Model | MMLU | GSM8K | HumanEval | Multilingual | MT-Bench |
|---|---|---|---|---|---|
| Mistral-7B | 64.1 | 47.5 | 27.4 | 40.0 | 7.60 |
| Gemma-7B | 64.6 | 50.9 | 32.3 | - | - |
| Qwen1.5-7B | 61.0 | 62.5 | 36.0 | 45.2 | 7.60 |
| DeepSeekMoE 16B | 45.0 | 18.8 | 26.8 | - | 6.93 |
| Qwen1.5-MoE-A2.7B | 62.5 | 61.5 | 34.2 | 40.8 | 7.17 |

| Model | #Parameters | #(Activated) Parameters |
|---|---|---|
| Mistral-7B | 7.2 | 7.2 |
| Qwen1.5-7B | 7.7 | 7.7 |
| Gemma-7B | 8.5 | 7.8 |
| DeepSeekMoE 16B | 16.4 | 2.8 |
| Qwen1.5-MoE-A2.7B | 14.3 | 2.7 |

第 4 讲 · 原页 19–21 · 中文讲解

# 专家并行与模型对比表的正确口径

## 原页 **19** 的跨设备箭头

图中普通注意力等部件可在各设备保留副本，专家则分布在不同设备。每个设备上的 token 经 router 后可能发往别处，专家算完再返回：这是 dispatch 与 combine 的两轮数据交换。它使总专家权重跨设备容纳，但引入 all-to-all 通信。如果只有一张卡且装不下总参数，稀疏激活并不会自动解决存储；若频繁从 CPU 调专家，传输延迟可能抵消算量收益。

## 原页 **20** 与 **21**：发布结果和参数表

原页 20 展示不同开源/开放权重模型的任务表现和成本信息，应视为课件时点的案例，不能据此推断所有开放模型当前排名或商业价格。原页 21 左表比较 MMLU、GSM8K、HumanEval 等任务，右表将 #Parameters 与 #Activated Parameters 分列。以课件 Qwen1.5-MoE-A2.7B 的 14.3B 总参数、2.7B 激活参数为例，名称中的 A2.7B 描述每 token 访问量，不是整个模型只有 2.7B 权重。不同任务列采用不同评分口径，不能直接把数字相加比较。

## 编者练习：同一批会用多少权重

即使每 token 只选 2/8 专家，四个 token 若分别选择 {1,2}、{3,4}、{5,6}、{7,8}，整批会触及全部 8 个专家。因此低 batch 解码和大 batch 训练的带宽、缓存复用、专家矩阵大小可能非常不同。比较部署预算时应同时报告总参数、激活参数、batch、上下文与设备分布，而不是只用激活参数代替所有资源指标。

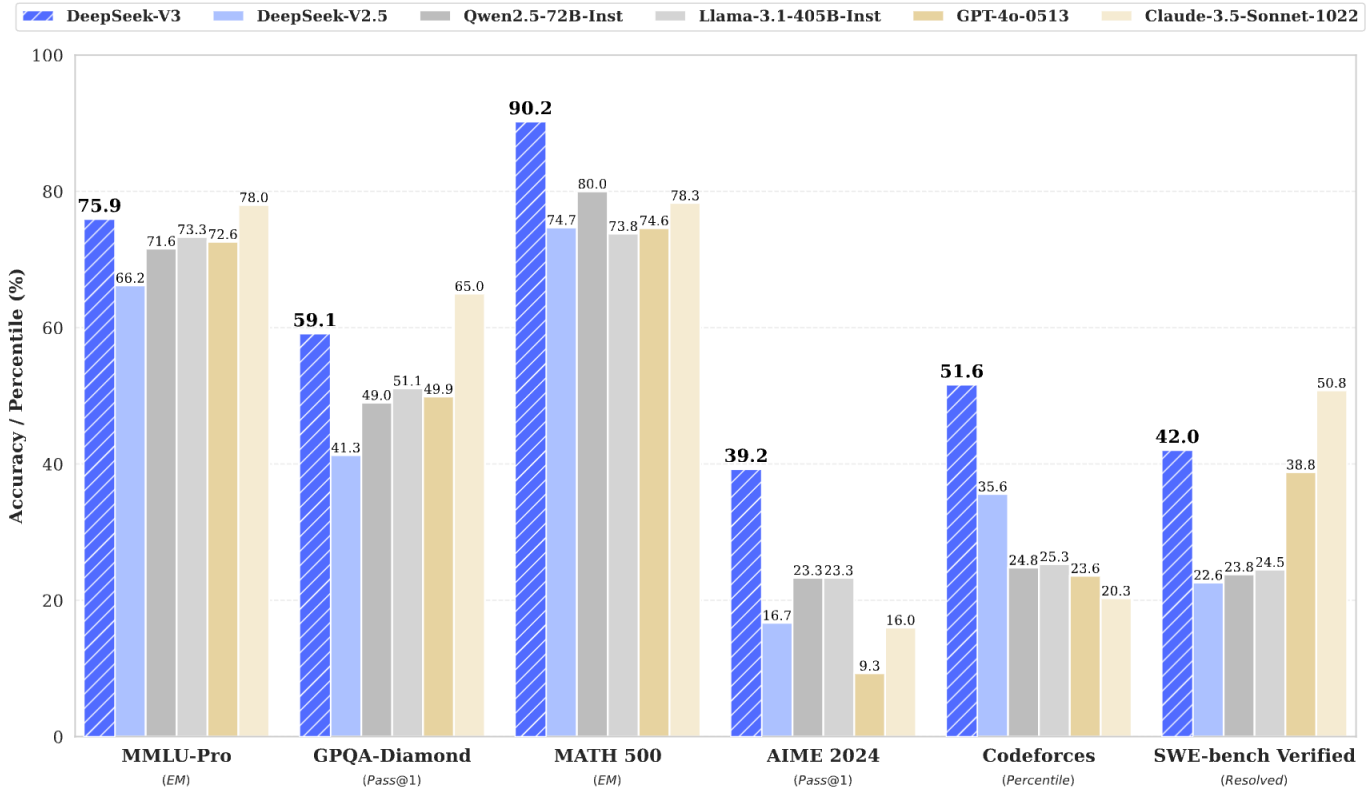
本节参考（完整书目见讲义末尾）：[slides]

# Earlier MoE results from Chinese groups - DeepSeek

There's also some good recent ablation work on MoEs showing they're generally good

| Metric | # Shot | Dense | Hash Layer | Switch |
|---|---|---|---|---|
| # Total Params | N/A | 0.2B | 2.0B | 2.0B |
| # Activated Params | N/A | 0.2B | 0.2B | 0.2B |
| FLOPs per 2K Tokens | N/A | 2.9T | 2.9T | 2.9T |
| # Training Tokens | N/A | 100B | 100B | 100B |
| Pile (Loss) | N/A | 2.060 | 1.932 | 1.881 |
| HellaSwag (Acc.) | 0-shot | 38.8 | 46.2 | 49.1 |
| PIQA (Acc.) | 0-shot | 66.8 | 68.4 | 70.5 |
| ARC-easy (Acc.) | 0-shot | 41.0 | 45.3 | 45.9 |
| ARC-challenge (Acc.) | 0-shot | 26.0 | 28.2 | 30.2 |
| RACE-middle (Acc.) | 5-shot | 38.8 | 38.8 | 43.6 |
| RACE-high (Acc.) | 5-shot | 29.0 | 30.0 | 30.9 |
| HumanEval (Pass@1) | 0-shot | 0.0 | 1.2 | 2.4 |
| MBPP (Pass@1) | 3-shot | 0.2 | 0.6 | 0.4 |
| TriviaQA (EM) | 5-shot | 4.9 | 6.5 | 8.9 |
| NaturalQuestions (EM) | 5-shot | 1.4 | 1.4 | 2.5 |

# Recent MoE results – DeepSeek v3



# Why haven't MoEs been more popular?

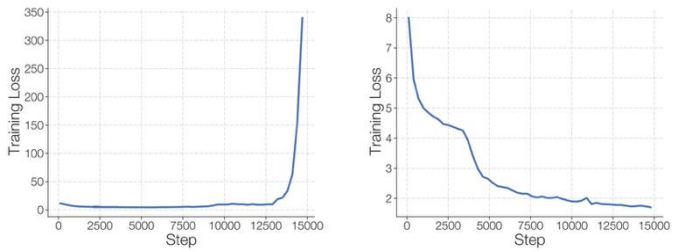
Infrastructure is complex / advantages on multi node

> At a high level, sparsity is good when you have many accelerators (e.g. GPU/TPU) to host all the additional parameters that comes when using sparsity. Typically models are trained using data-parallelism where different machines will get different slices of the training/inference data. The machines used for operating on the different slices of data can now be used to host many more model parameters. Therefore, sparse models are good when training with data parallelism and/or have high throughput while serving: training/serving on many machines which can host all of the parameters.

[Fedus et al 2022]

Training objectives are somewhat heuristic (and sometimes unstable)

Sparse models often suffer from training instabilities (Figure 1) worse than those observed in standard densely-activated Transformers.




[Zoph et al 2022]

第 4 讲 · 原页 22–24 · 中文讲解

# 证据与代价：MoE 效果好仍不等于免费

## 原页 22 的受控比较

表中 Dense、Hash Layer、Switch 的每 token 激活参数、训练 token 与 FLOPs 对齐，而总参数不同。多数任务与 Pile loss 显示稀疏模型有收益，这比跨公司表格更能支持“同激活计算下增加可选参数有用”。同时 Hash 也是稀疏模型，Switch 相对 Hash 的收益才更接近学到路由本身的作用。逐列读指标方向：loss 越低越好，accuracy 越高越好，不能把所有列都按越大越好看。

## 原页 23：大规模结果是组合系统的成绩

DeepSeek-V3 柱图展示数学、代码、通识等基准，而模型同时使用数据策略、MLA、MoE、训练稳定性与后训练改进。它证明这套组合可以达到很强表现，不能用此图把收益精确拆给某个专家数或均衡算法。后续原页 54–59 会拆开架构部件；学习时应该先理解每个部件改变的成本，再看整套系统的实证结果。

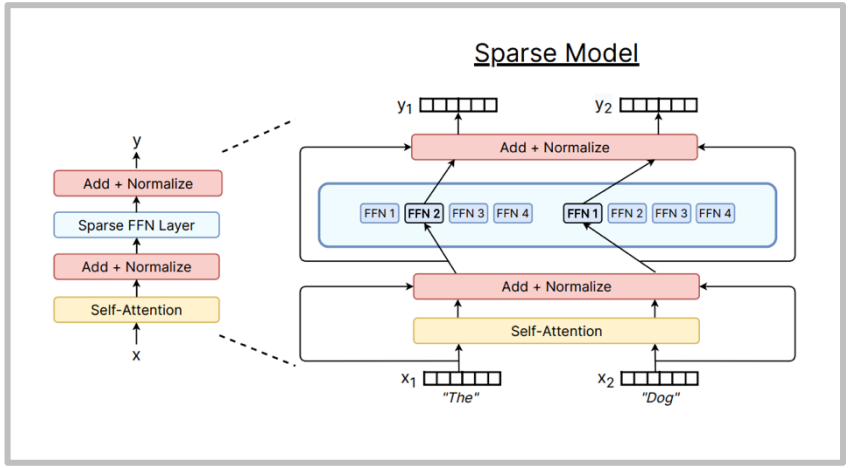
## 原页 24 的两类障碍

上方摘录强调稀疏模型需要足够多的加速器承载额外权重和高效通信；下方训练损失尖峰说明路由与数值稳定性也可能成为问题。稀疏带来不规则工作量：热门专家排队，冷门专家空闲，负载分布还随训练变化。于是一个好用的 MoE 不只是 FFN 列表加 top-k，还需要容量管理、稳定 router、通信实现与训练目标配合。后半讲将按“选谁、专家多大、怎样训练”展开这些问题。

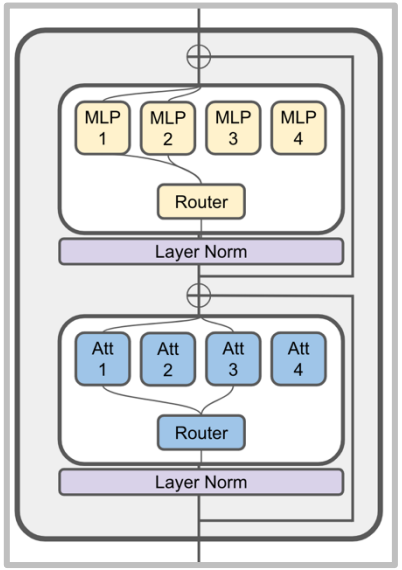
本节参考（完整书目见讲义末尾）：[slides]

# What MoEs generally look like

Typical: replace MLP with MoE layer




Less common: MoE for attention heads




[ModuleFormer, JetMoE]

# MoE – what varies?

- Routing function
- Expert sizes
- Training objectives

第 4 讲 · 原页 25–26 · 中文讲解

# MoE 设计的三个自由度

## 替换哪个模块

原页 25 左图是常见做法：保留序列混合用的注意力，把逐 token MLP 改成 MoE。这样 router 接收每个 token 表示，不直接决定这个 token 看哪个历史位置。右图给出注意力头也做专家化的架构；它说明 MoE 是条件计算思想，不局限于 FFN，但也会涉及注意力状态和通信的额外设计。本讲主体聚焦 FFN 专家，不能把右图的细节当成后续所有模型都采用的默认。

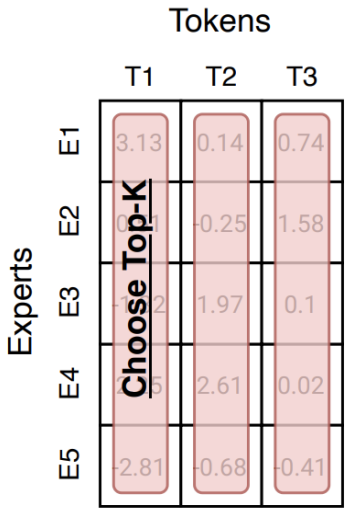
## 把三个问题分开思考

路由函数决定 token 与专家的分配关系，以及输出权重如何归一化；专家大小决定每次选择的粒度与总参数；训练目标决定如何兼顾任务损失、负载和数值稳定性。三者相互约束，例如拆小专家会增加可组合选择，同时让路由表更宽、矩阵更小；提高 k 会增加每 token 访问容量，也增加主算量和通信。之后看到模型名或专家总数，先补齐这三个问题，避免只凭“有 256 个专家”推断质量和速度。

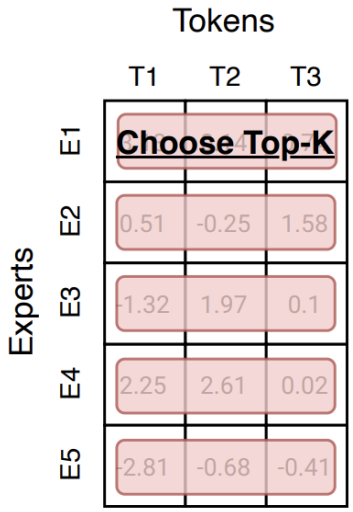
本节参考（完整书目见讲义末尾）：[slides]

# Routing function - overview

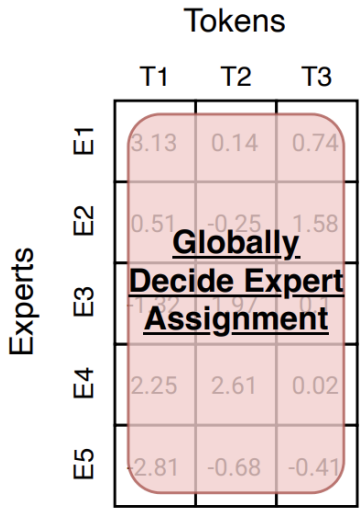
Many of the routing algorithms boil down to ‘choose top k’




Token chooses expert




Expert chooses token

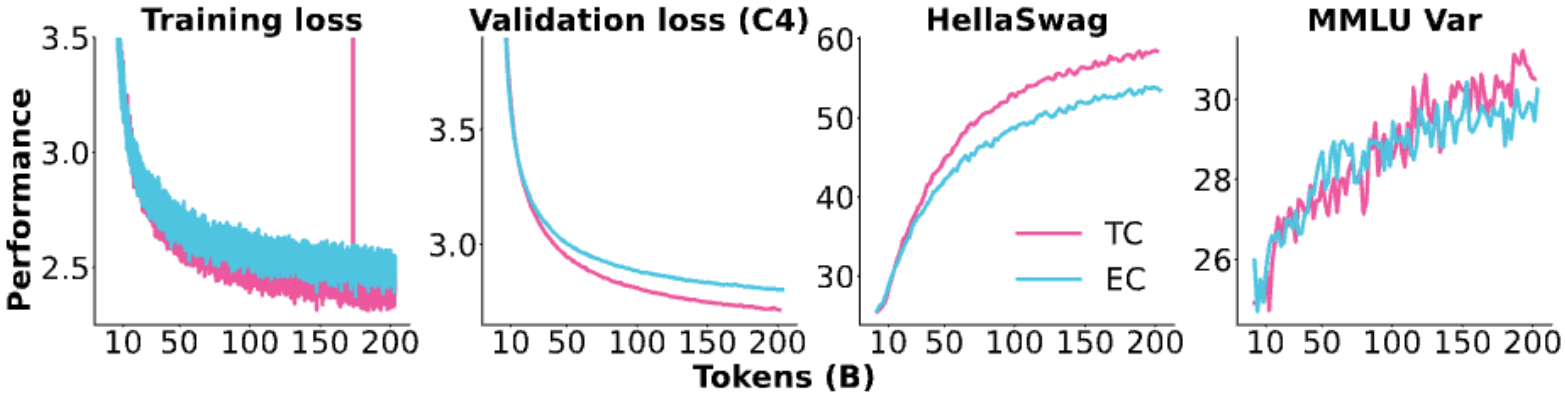



Global routing via optimization

[Fedus et al 2022]

# Routing type

Almost all the MoEs do a standard ‘token choice topk’ routing. Some recent ablations



第 4 讲 · 原页 27–28 · 中文讲解

# 谁选择谁：token choice、expert choice 与受控比较

## 读原页 27 的分数矩阵

图中行是专家、列是 token，每个格子是配对分数。token choice 沿每一列选 top-k，每个 token 获得固定数量的专家，但每行被选次数不确定；expert choice 沿每一行选择固定数量的 token，专家负载受控，但每个 token 可能被不同数量的专家接收。全局路由则同时考虑容量与总分，求一个约束分配。三个面板用同一分数表说明“top-k”必须说清沿哪个维度做。

## 编者例子：局部最佳与负载约束的冲突

设 3 个 token 的两专家分数依次为 (0.9,0.1)、(0.8,0.2)、(0.7,0.3)。token choice 的 top-1 全选专家 A；若 A 容量只有 2，则必须丢弃、重路由或增加容量。若每专家独立选 2 个 token，A 选前两个，B 选后两个，中间 token 被选两次，其他各一次；不再是每 token 恰好一个专家。若全局要求 A 接 2 个、B 接 1 个，最佳方案把第三个给 B，总分 0.9+0.8+0.3=2.0。最优局部选择与系统负载不是同一个目标。

## 训练与因果性的提醒

一个 token 的 router 通常只看当前层该 token 的表示，但 batch 层面的容量约束会让最终分配受其他 token 影响。expert choice 的跨 token 竞争在自回归训练中还需要特别设计，不能未经检查直接借助未来 token 的分数来决定过去输出。固定 token choice 更容易保持每个 token 的确定计算预算，但负载问题必须在后面另行处理。

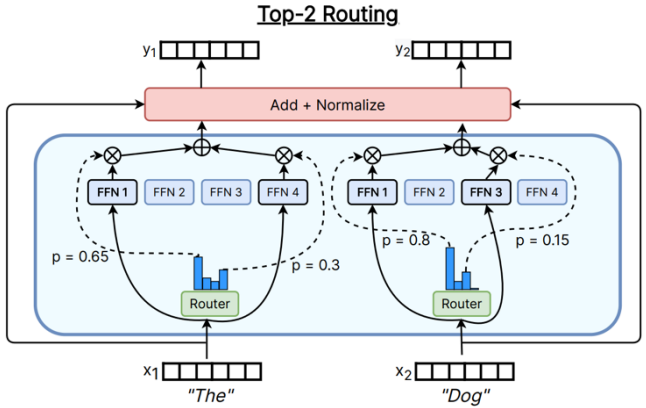
## 原页 28 的曲线

OLMoE 图把 token choice（TC）与 expert choice（EC）置于同一组训练 token 横轴下，分别比较训练损失、C4 验证损失、HellaSwag 与 MMLU。在该配方中 TC 的综合表现更有利；仍应逐项看指标方向，并确认每 token 计算预算对齐，不能只看某个训练时刻的一条线就推出所有模型都应使用 TC。这个对比展示路由方向不仅影响负载，还可能改变学习行为。

本节参考（完整书目见讲义末尾）：[slides]

# Common routing variants in detail

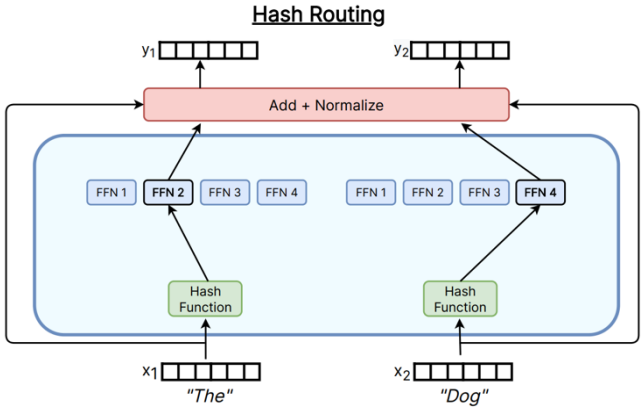
Top-k




Used in *most* MoEs

Switch Transformer (k=1)
Gshard (k=2), Grok (2), Mixtral (2),
Qwen (4), DBRX (4),
DeepSeek (7)

Hashing

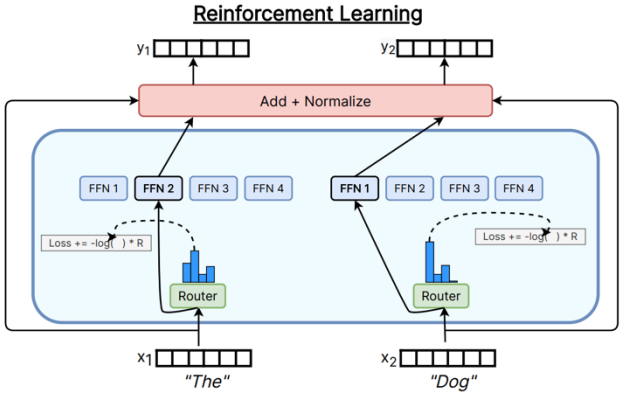



Common baseline

[Fedus et al 2022]

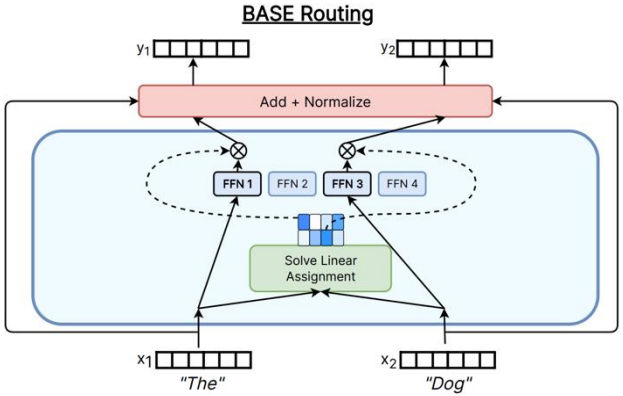
# Other routing methods

RL to learn routes




Used in some of the earliest work
Bengio 2013, not common now

Solve a matching problem




Linear assignment for routing
Used in various papers like Clark ‘22

[Fedus et al 2022]

第 4 讲 · 原页 29–30 · 中文讲解

# 路由方法的选择：top-k、哈希、策略与匹配

## 原页 **28–30** 的方法与实验边界

原页 28 的 OLMoE 曲线对比 token choice 与 expert choice，在其设置下前者的损失与下游表现更有利；这不是 expert choice 永远较差的定理。原页 29 的 hashing 按固定规则映射，省去学路由但适应性较弱；top-k 则由表示学习打分。页中列出的 k 对应特定版本，不能把“Qwen=4”或“DeepSeek=7”当成整个模型家族永久定义，尤其要区分共享专家是否混入计数。原页 30 的强化学习直接学离散策略，匹配方法解全局约束分配；它们需要额外优化成本或训练技巧。

## 编者对比：用什么代价换什么能力

top-k 基于当前 token 的表示，能够随上下文改变专家选择，但硬排名有梯度难点。哈希可把 token ID 或其他键确定地映射到专家，简单、稳定且适合作基线，但本身不通过任务损失学语义分配，固定词频也可能造成负载偏斜。强化学习将路由当离散动作，直接优化期望任务表现，代价是采样方差与训练复杂度。全局匹配在一个 batch 内优化总分并遵守容量约束，能控制负载，却需要额外求解且存在批次耦合。后面几页集中讲最常见的 top-k，不表示其余方法在理论上错误。

本节参考（完整书目见讲义末尾）：[slides]

# Top-K routing in detail.

*Most* papers do the old and classic top-k routing. How does this work?

Gating

$$\mathbf{h}_t^l = \sum_{i=1}^{N} \left( g_{i,t} \, \mathrm{FFN}_i \left( \mathbf{u}_t^l \right) \right) + \mathbf{u}_t^l,$$

$$g_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} \in \mathrm{Topk}(\{s_{j,t} | 1 \leqslant j \leqslant N\}, K), \\ 0, & \text{otherwise}, \end{cases}$$

$$s_{i,t} = \mathrm{Softmax}_i \left( \mathbf{u}_t^{l\,T} \mathbf{e}_i^l \right),$$

Gates selected by a logistic regressor

This is the DeepSeek (V1-2) router (Grok, Qwen do this too)

Mixtral, DBRX, DeepSeek v3 softmaxes after the TopK

[Dai et al 2024]

第 4 讲 · 原页 31–31 · 中文讲解

# Top-k 路由：选择集合与混合权重是两步

## 符号与前向过程

对当前 token 的层输入 u_t，令 a_i=u_t$^{T}$e_i，其中 e_i 是 router 为专家 i 学到的向量。分数经过 softmax 得 s_i，再选择集合 I=TopK(s,k)，设置 g_i=s_i（i∈I）或 0。专家输出 o_i=FFN_i(u_t)，最终 h_t=u_t+Σ_i g_i o_i，u_t 是残差，不能在统计专家时把它算成另一个专家。原页把 gate 叫 logistic regressor，是指线性打分加概率映射，专家本身仍是非线性 FFN。

## 编者手算：先归一化再截断与选中后归一化

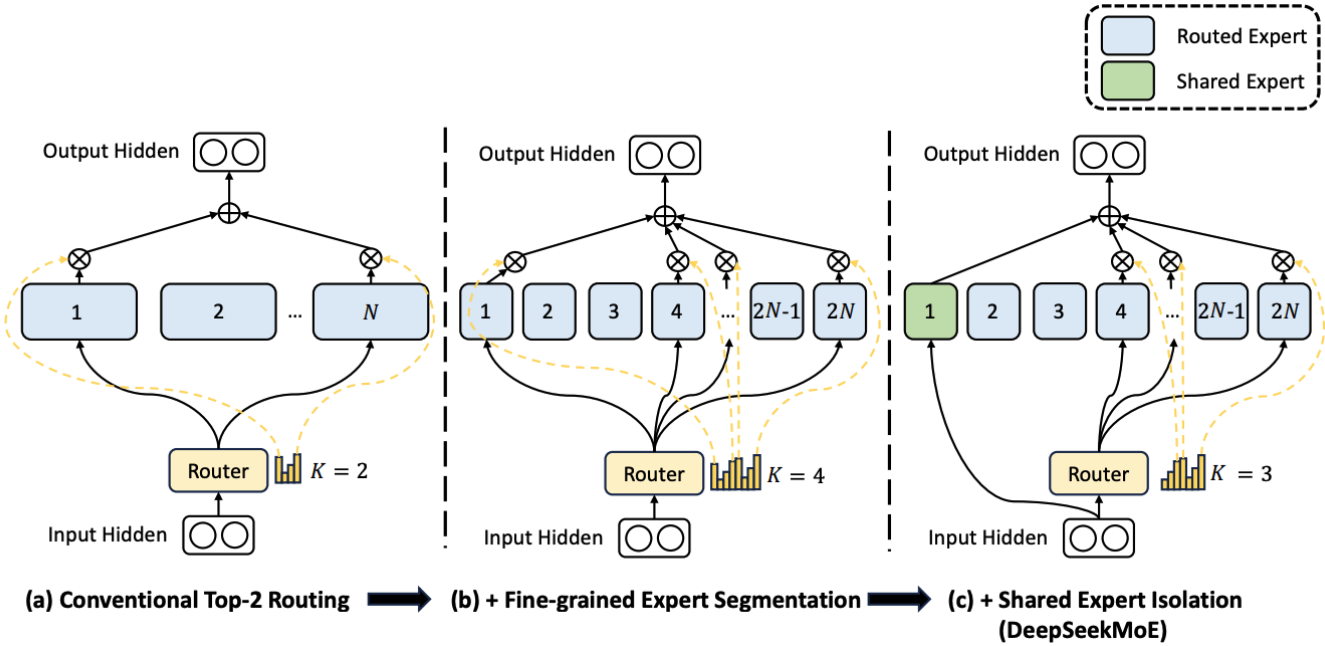
取全体 softmax 概率为 (0.6,0.3,0.1)，k=2，选中的两个专家输出为 10、20。先 softmax 再置零且不重归一化，输出是 0.6×10+0.3×20=12。只对选中 logits 做 softmax，则权重为 (2/3,1/3)，输出为 40/3≈13.33。若没有额外偏置或分组约束，由于 softmax 保序，两者选择集合相同，但输出尺度和梯度不一样。k=1 时，选中后归一化的权重恒为 1，对硬选择本身没有普通导数；保留全体 softmax 的选中概率则仍有门权重梯度。

## 原稿核对：**DeepSeek-V3** 的特殊情况

原页将 V3 与“softmaxes after TopK”放在一起，是过度简写。V3 技术报告中 affinity 为 sigmoid(a_i)，选中后用这些分数除以选中分数之和；不是把 sigmoid 分数再送进 softmax。其负载 bias 影响入选集合，不直接加进最终混合权重。原页不改，实际实现应按原页 42、56 的中文展开和论文公式核对。比如选中分数 0.8、0.4，经和归一化是 2/3、1/3，而 softmax(0.8,0.4) 约为 0.599、0.401，两者不同。

本节参考（完整书目见讲义末尾）：[slides] · [deepseek3]

# Recent variations from DeepSeek and other Chinese LMs




Smaller, larger number of experts + a few shared experts that are always on.

(Used in DeepSeek / Qwen, originally from DeepSpeed MoE)

# Various ablations from the DeepSeek paper

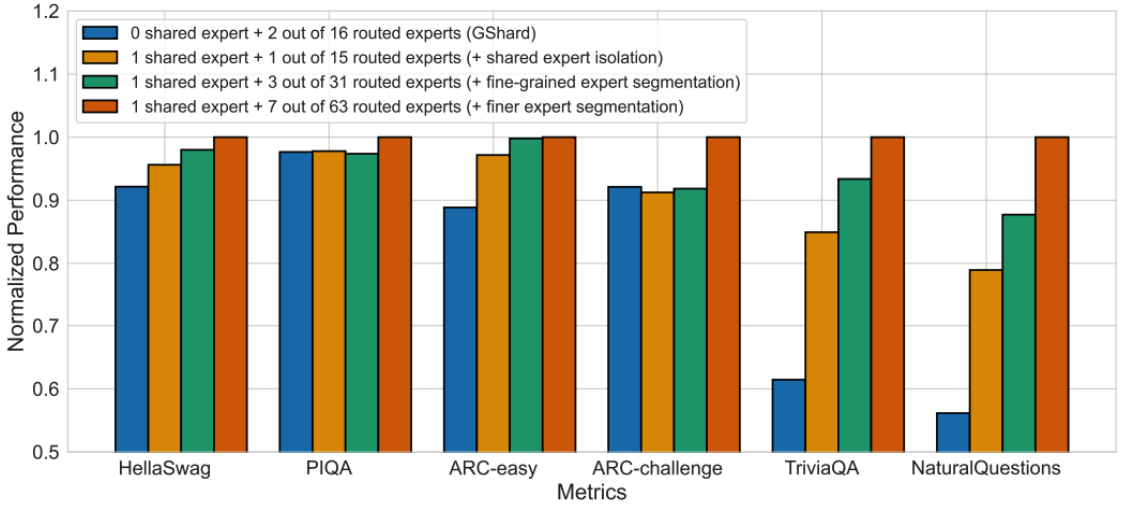
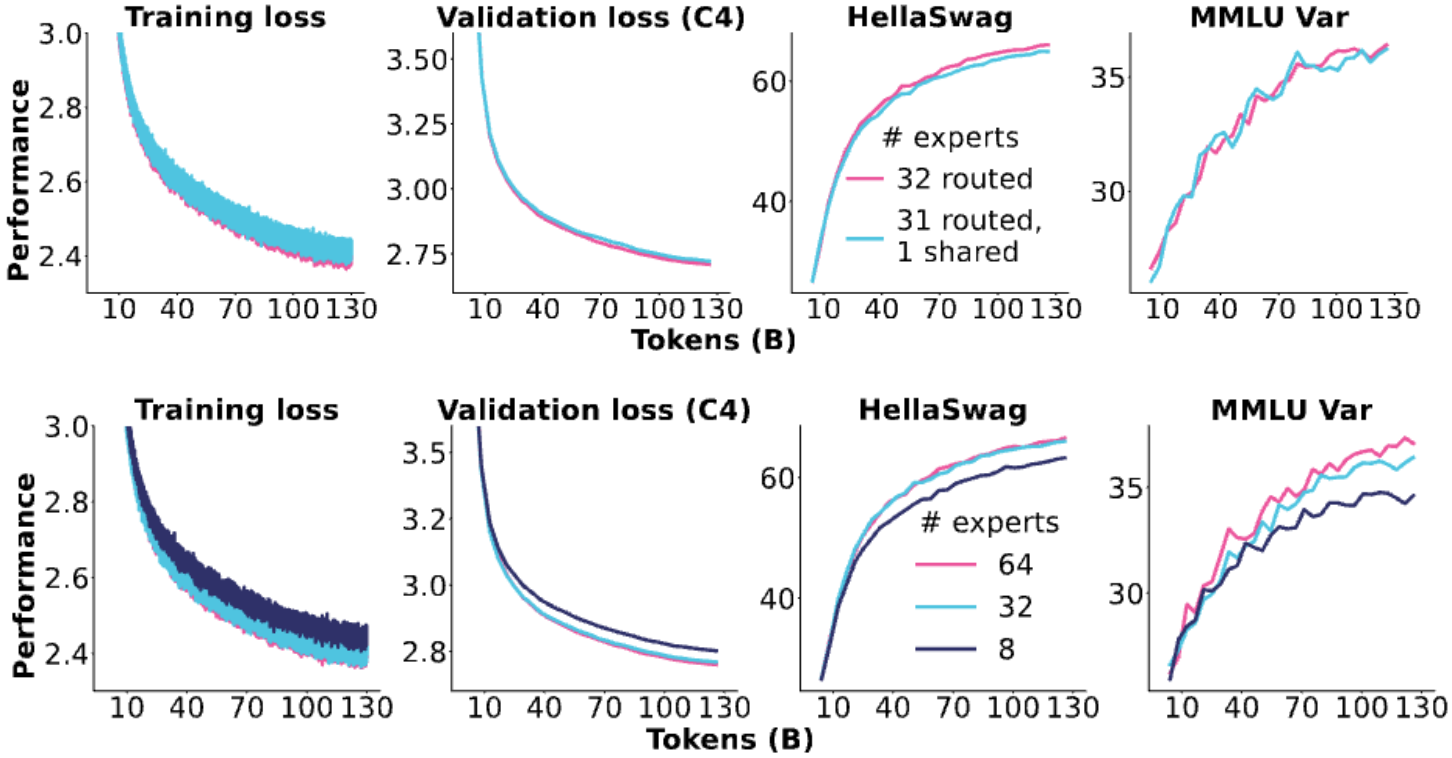



Figure 3 | Ablation studies for DeepSeekMoE. The performance is normalized by the best performance for clarity in presentation. All compared models have the same number of parameters and activated parameters. We can find that fine-grained expert segmentation and shared expert isolation both contribute to stronger overall performance.

More experts, shared experts all seem to generally help

# Ablations from OlMoE

Gains from fine-grained experts, none from shared experts.



第 4 讲 · 原页 32–34 · 中文讲解

# 专家粒度与共享专家：组合更多，收益需消融

## 原页 **32** 的三幅结构图

传统 top-2 在较大的专家中选两个；细粒度拆分把每个 FFN 的中间维度缩小，专家数量增多，每 token 可选更多小专家，保持激活宽度近似不变。共享专家是始终执行的 FFN，处理所有输入，剩余路由专家提供条件选择。图中绿色块表示共享，蓝色块表示可路由；共享不等于各设备复制的权重副本，也不等于不同层共享同一组权重。

## 编者例子：为什么细粒度能改变表达组合

设 8 个宽度 m 的专家，选 2 个，主要激活宽度为 2m，总专家宽度为 8m。改成 16 个宽度 m/2 的专家，选 4 个，两个预算仍分别为 2m 与 8m；选择组合从 C(8,2)=28 增到 C(16,4)=1820。组合数增加不是模型质量的证明，但说明相同参数/主算量下可更细致地拼接功能。代价是路由更宽、分发条数增加、小矩阵效率可能下降，不能只看组合数。共享专家若占用同样预算，会减少自由路由部分的组合。

## 原页 **33** 与 **34** 的不一致是重要结论

DeepSeek 的受控柱图中，细粒度和共享专家的组合改善了若干指标；OLMoE 上方曲线比较有无共享专家，未见相同稳定收益，下方则显示细粒度更有利。这两个结果可以同时成立：规模、专家宽度、数据、训练时间、路由及均衡限制不同。共享专家“吸收共同知识、让其他专家更专门化”是设计动机，不是强制实现的语义分工。这里最应学到的是提出可检验假设，并在相同总参数、激活参数和训练预算下比较。

本节参考（完整书目见讲义末尾）：[slides] · [deepseek-moe] · [olmoe]

# Expert routing setups for recent MoEs

| Model | Routed | Active | Shared | Fine-grained ratio |
|---|---|---|---|---|
| GShard | 2048 | 2 | 0 | |
| Switch Transformer | 64 | 1 | 0 | |
| ST-MOE | 64 | 2 | 0 | |
| Mixtral | 8 | 2 | 0 | |
| DBRX | 16 | 4 | 0 | |
| Grok | 8 | 2 | 0 | |
| DeepSeek v1 | 64 | 6 | 2 | 1/4 |
| Qwen 1.5 | 60 | 4 | 4 | 1/8 |
| DeepSeek v3 | 256 | 8 | 1 | 1/14 |
| OlMoE | 64 | 8 | 0 | 1/8 |
| MiniMax | 32 | 2 | 0 | ~1/4 |
| Llama 4 (maverick) | 128 | 1 | 1 | 1/2 |

第 4 讲 · 原页 35–35 · 中文讲解

# 读专家配置表：E、k、共享数量与粒度

## 每一列的含义

Routed 是可路由专家总数 E，Active 是每 token 选中的路由专家数 k，Shared 是始终启用的专家数 N_s；Fine-grained ratio 描述相对某个常规 FFN 的宽度比例。比例的参考 FFN 随模型而异，所以不能把不同模型同列直接当成统一参数单位。原页表里 Mixtral 的 8/2/0 与 DeepSeek-V3 的 256/8/1 表示路由结构不同，并不意味着后者每 token 算量必为前者四倍。还要乘上专家宽度，加入共享部分以及各模型的注意力和层数。

## 编者手算：单层专家量与全模型不能混算

若某模型每个细粒度专家只有参考 FFN 的 1/14，则 8 个路由加 1 个共享的激活专家主参数约为 9/14 个参考 FFN，总专家主参数约为 257/14 个参考 FFN。这只解释该层专家部分；不是全模型 37B/671B 的直接计算公式。整个模型还含注意力、embedding、普通稠密层、router 等。报告“激活参数”还应说明是否包含 embedding/output head 以及 MTP 模块，避免不同论文统计口径混用。

## 阅读版本标记

本表是课程收集的架构实例，不是实时配置索引。原页 29 的 DeepSeek(7)、此页的 v1 6 路由加 2 共享、v3 8 路由加 1 共享属于不同版本或简写，不能拼成一份统一配置。第 56 页中文会按 V3 报告处理 258 的笔误，以此页 256 routed 的信息交叉核对。

本节参考（完整书目见讲义末尾）：[slides]

# How do we train MoEs?

**Major challenge:** we need sparsity for training-time efficiency…

But sparse gating decisions are not differentiable!

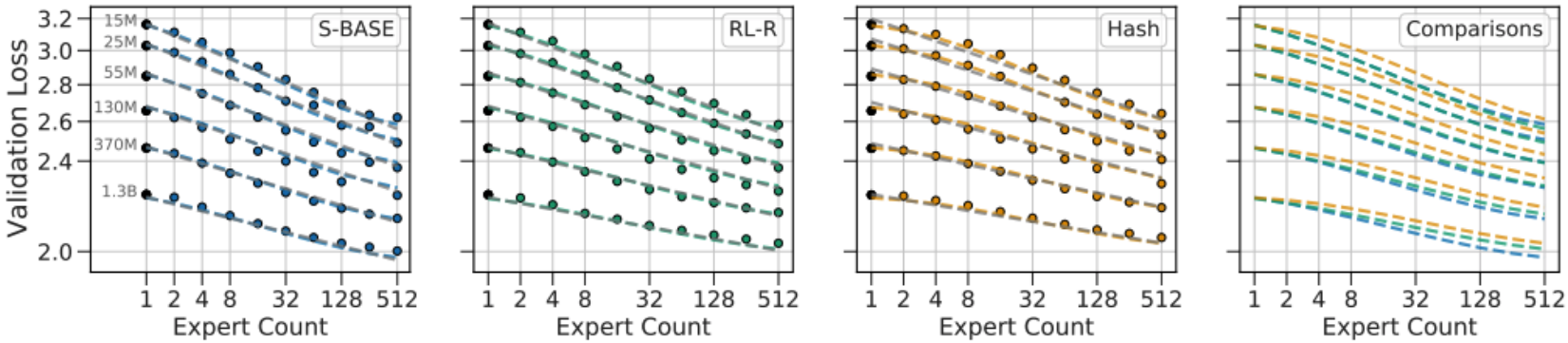
**Solutions?**

1. Reinforcment learning to optimize gating policies
2. Stochastic perturbations
3. Heuristic ‘balancing’ losses.

Guess which one people use in practice?

# RL for MoEs

RL via REINFORCE does work, but not so much better that it's a clear win




(REINFORCE baseline approach, Clark et al 2020)

RL is the 'right solution' but gradient variances and complexity means it's not widely used

第 4 讲 · 原页 36–37 · 中文讲解

# 离散路由怎样学习：任务梯度与策略梯度

## 困难究竟在哪里

如果把输出写成 Σ_i 1[i∈TopK(a)]g_i FFN_i(x)，专家网络和选中分支上的连续 gate 可以反向传播；难点是集合成员随分数排名发生跳变。远离边界时硬选择指示函数局部不变，跨越边界时不连续，普通导数无法直接表达“换一个专家可能更好”的效果。不能因此说 MoE 完全不可微，也不能说 softmax 一加上就让 top-k 可微。不同 gate 归一化方法，尤其 k=1 时，会影响任务损失能否给 router 连续梯度。

## 编者补充：**REINFORCE** 的含义

若从策略 π_θ(i|x) 采样专家，目标 J=E_i[L(i)]，则路由部分梯度可用 E_i[(L(i)−b)∇_θ log π_θ(i|x)] 估计，b 是降低方差的 baseline，不能随当前采样动作以破坏无偏性的方式选取。损失更低的选择会得到相应鼓励，但单次采样只看到所选专家结果，因此估计噪声大。它处理离散决策有原则依据，却增加方差控制与工程复杂度；若专家参数也依赖 θ，还需另外计算分支内部的路径梯度。

## 原页 **37** 的实验怎么读

横轴专家数、纵轴验证损失，多条线对应不同模型规模；RL 路由在一些设置中可用，但并未表现出足以抵消复杂度的普遍优势。课件把 RL 称作“right solution”应理解为解决离散优化的一条原则性路线，而非唯一正确算法。实际常见方案仍是连续 gate 的任务梯度配合扰动和均衡手段，接下来几页分别解释这些补偿机制。

本节参考（完整书目见讲义末尾）：[slides]

# Stochastic approximations

$$G(x) = Softmax(KeepTopK(H(x), k))$$

$$H(x)_i = (x \cdot W_g)_i + StandardNormal() \cdot Softplus((x \cdot W_{noise})_i)$$

$$KeepTopK(v, k)_i = \begin{cases} v_i & \text{if } v_i \text{ is in the top } k \text{ elements of } v. \\ -\infty & \text{otherwise.} \end{cases}$$

From Shazeer et al 2017 – routing decisions are *stochastic* with gaussian perturbations.

1. This naturally leads to experts that are a bit more robust.
2. The softmax means that the model learns how to rank K experts

# Stochastic approximations

```
if is_training:
    # Add noise for exploration across experts.
    router_logits += mtf.random_uniform(shape=router_logits.shape, minval=1-eps, maxval=1+eps)

# Convert input to softmax operation from bfloat16 to float32 for stability.
router_logits = mtf.to_float32(router_logits)

# Probabilities for each token of what expert it should be sent to.
router_probs = mtf.softmax(router_logits, axis=-1)
```

Stochastic jitter in Fedus et al 2022. This does a uniform multiplicative perturbation for the same goal of getting less brittle experts. This was later removed in Zoph et al 2022

| Method | Fraction Stable | Quality (↑) |
|---|---|---|
| Baseline | 4/6 | **-1.755** ±0.02 |
| Input jitter ($10^{-2}$) | 3/3 | -1.777 ±0.03 |
| Dropout (0.1) | 3/3 | -1.822 ±0.11 |

第 4 讲 · 原页 38–39 · 中文讲解

# 随机扰动：探索路由，不能替代均衡

## 读原页 **38** 的 **noisy top-k** 公式

先计算 H_i(x)=(xW_g)_i+ε_i softplus((xW_noise)_i)，ε_i 为标准高斯噪声；softplus 使噪声标准差为正。KeepTopK 保留最高的 k 个值，其他设为 −∞，再 softmax，让未选专家权重为 0。若将其他位置设成 0，则 exp(0)=1，会让未选专家仍拿到概率，完全不是同一算法。噪声让临界位置有机会尝试不同专家，降低早期偶然分配被固化的风险。

## 编者例子与条件

若两个专家 logits 为 1.00 和 0.99，微小扰动能交换名次；若为 5 和 −5，同量级小噪声几乎不改变选择。因此噪声在决策边界附近作用最强，不能保证所有专家自动均匀获客。训练时采样、推理时关闭还会形成行为差异，需要在配方中明确。top-k 的成员依然是离散的，给 logits 加噪声不等于得到了硬 top-k 的普通精确梯度。

## 原页 **39** 的代码与消融

这里的 jitter 是均匀乘法扰动，和上一页高斯加法不是同一种噪声：乘数在 [1−ε,1+ε]，扰动幅度与被乘值相关。课件代码随后把 router 的 softmax 转为 float32，这是数值精度措施，也不是另一种探索噪声。下表比较稳定运行比例与质量，后续 ST-MoE 配方并未继续依赖同样的 jitter。应把它视作需消融的训练技巧，不能写成所有 MoE 稳定训练的必要条件。

本节参考（完整书目见讲义末尾）：[slides]

# Heuristic balancing losses

Another key issue – systems efficiency requires that we use experts evenly..

For each Switch layer, this auxiliary loss is added to the total model loss during training. Given $N$ experts indexed by $i = 1$ to $N$ and a batch $\mathcal{B}$ with $T$ tokens, the auxiliary loss is computed as the scaled dot-product between vectors $f$ and $P$,

$$\text{loss} = \alpha \cdot N \cdot \sum_{i=1}^{N} f_i \cdot P_i \tag{4}$$

where $f_i$ is the fraction of tokens dispatched to expert $i$,

$$f_i = \frac{1}{T} \sum_{x \in \mathcal{B}} \mathbb{1}\{\text{argmax } p(x) = i\} \tag{5}$$

and $P_i$ is the fraction of the router probability allocated for expert $i$, [2]

$$P_i = \frac{1}{T} \sum_{x \in \mathcal{B}} p_i(x). \tag{6}$$

From the Switch Transformer [Fedus et al 2022]

The derivative with respect to $p_i(x)$ is $\frac{\alpha N}{T^2} \sum 1_{argmax\ p(x)=i}$,
so more frequent use = stronger downweighting

第 4 讲 · 原页 40–40 · 中文讲解

# 负载均衡损失：从统计量到 logits 梯度

## 定义：硬分配比例和软概率均值

以 Switch 的 top-1 路由为例，batch 有 T 个 token、N 个专家。$f_i=(1/T)\Sigma_x 1[\text{argmax}_j p_j(x)=i]$ 是实际选择专家 i 的 token 比例，$P_i=(1/T)\Sigma_x p_i(x)$ 是 router 给它的平均概率。$L_{bal}=\alpha N\Sigma_i f_i P_i$，α 是辅助损失系数。两组量不同：f 计数离散路由，P 保留连续可微分数。若两者都均匀，则 $f_i=P_i=1/N$，$L_{bal}=\alpha$；若两者都集中在同一专家，则约为 αN。此例说明设计意图，不应声称该带离散统计的函数在所有配置下都具有唯一均匀全局极小值。

## 编者推导：热门专家为什么被下压

反向传播通常把 $f_i$ 当停止梯度的统计量。于是 $\partial L_{bal}/\partial P_i=\alpha N f_i$，再由 $P_i$ 对某 token 概率的导数 1/T，得 $\partial L_{bal}/\partial p_i(x)=\alpha N f_i/T$。它为正，且频次越高越大，梯度下降倾向减少热门专家概率。但 softmax 概率不能独立变化；若 logits 为 $a_i$，完整链式法则给出 $\partial L_{bal}/\partial a_i(x)=(\alpha N/T)p_i(x)[f_i-\Sigma_j p_j(x)f_j]$。高于该 token 加权平均频次的专家被压低，低于平均者得到相对提升。

## 编者手算：负载偏斜的信号

取 N=2、T=4，实际选择为 3 个到 A、1 个到 B，所以 f=(0.75,0.25)；平均概率 P=(0.6,0.4)，则 $L_{bal}=2\alpha(0.75\times0.6+0.25\times0.4)=1.1\alpha$。对某个概率为 (0.6,0.4) 的 token，概率梯度分别为 0.375α、0.125α，logit 梯度为 (0.06α,−0.06α)。后者和为零，反映 softmax 对共同平移不敏感。负载损失不会神奇地给每个未选专家任务反馈，它只是为 router 提供分配压力。

## 为什么不能把均衡系数越调越大

训练目标通常是 $L_{task}+L_{bal}$+其他正则。过强均衡可能强迫语义上很不合适的 token 去冷门专家；过弱则导致热门专家溢出、其他参数缺乏训练和设备空闲。均衡是系统与学习之间的折中，而非保证每个样本都平均分给全部专家。top-k>1 的计数需按 kT 或论文规定的比例归一化，不能原样套 top-1 的 f 定义而忽略多次分配。

本节参考（完整书目见讲义末尾）：[slides] · [switch]

# Example from deepseek (v1-2)

**Per-expert balancing** – same as the switch transformer

$$\mathcal{L}_{\text{ExpBal}} = \alpha_1 \sum_{i=1}^{N'} f_i P_i, \tag{12}$$

$$f_i = \frac{N'}{K'T} \sum_{t=1}^{T} \mathbb{1}(\text{Token } t \text{ selects Expert } i), \tag{13}$$

$$P_i = \frac{1}{T} \sum_{t=1}^{T} s_{i,t}, \tag{14}$$

**Per-device balancing** – the objective above, but aggregated by device.

$$\mathcal{L}_{\text{DevBal}} = \alpha_2 \sum_{i=1}^{D} f'_i P'_i, \tag{15}$$

$$f'_i = \frac{1}{|\mathcal{E}_i|} \sum_{j \in \mathcal{E}_i} f_j, \tag{16}$$

$$P'_i = \sum_{j \in \mathcal{E}_i} P_j, \tag{17}$$

# DeepSeek v3 variation – per-expert biases

Set up a per-expert bias (making it more likely to get tokens) and use online learning

$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} + b_i \in \text{Topk}(\{s_{j,t} + b_j | 1 \leqslant j \leqslant N_r\}, K_r), \\ 0, & \text{otherwise.} \end{cases}$$

They call this '**auxiliary loss free balancing**'

**Complementary Sequence-Wise Auxiliary Loss.** Although DeepSeek-V3 mainly relies on the auxiliary-loss-free strategy for load balance, to prevent extreme imbalance within any single sequence, we also employ a complementary sequence-wise balance loss:

$$\mathcal{L}_{\text{Bal}} = \alpha \sum_{i=1}^{N_r} f_i P_i, \tag{17}$$

$$f_i = \frac{N_r}{K_r T} \sum_{t=1}^{T} \mathbb{1}\left(s_{i,t} \in \text{Topk}(\{s_{j,t} | 1 \leqslant j \leqslant N_r\}, K_r)\right), \tag{18}$$

$$s'_{i,t} = \frac{s_{i,t}}{\sum_{j=1}^{N_r} s_{j,t}}, \tag{19}$$

$$P_i = \frac{1}{T} \sum_{t=1}^{T} s'_{i,t}, \tag{20}$$

(but the approach is not fully aux loss free..)

第 4 讲 · 原页 41–42 · 中文讲解

# 设备均衡与 V3 的在线路由偏置

## 原页 41：专家均衡不完全等于设备均衡

DeepSeek v1–v2 同时考虑每专家频次与每设备聚合频次。设备上可能存多个专家，即使每个专家的平均概率看起来合理，某设备接收的 token 合计仍可能偏多。设 4 个专家分到两台设备，专家比例为 (0.3,0.3,0.2,0.2)，第一台接前两个时负载为 0.6，第二台为 0.4；改变专家放置也会改变瓶颈。页中 f 的归一化常含 N、k、T 的不同尺度，阅读公式时要先看它是否以 1 为均匀目标，不能把它直接与上一页取值和为 1 的 f 混同。

## 原页 42：把负载控制移到选择分数

令 s_i 为 token–expert affinity，b_i 为专家 bias，用 s_i+b_i 选 top-k，但输出使用未加 bias 的 s_i 归一化。训练中观测负载：拥挤专家减小 b_i，空闲专家增大 b_i。它类似一个反馈控制器，改变下次谁更容易入选，避免用强辅助损失把任务梯度持续推向均匀。bias 的更新速率需适当，过大可能造成分配振荡。

## 编者例子：bias 不改变专家的混合得分本身

两个专家 s=(0.7,0.6)，原本 top-1 选 A；若 b=(−0.2,0)，选择分数为 (0.5,0.6)，就改选 B。top-2 时，若两者均被选，最终权重按原始 s 得 (7/13,6/13)，不是按加 bias 后的 (5/11,6/11)。这分开了“谁可以执行”和“执行后贡献多少”。V3 仍保留序列级辅助均衡项来限制单条序列的极端偏斜，所以 auxiliary-loss-free 指主要专家级控制策略，不能译为训练目标完全没有任何辅助损失。

本节参考（完整书目见讲义末尾）：[slides] · [deepseek2] · [deepseek3]

# What happens when removing load balancing losses?

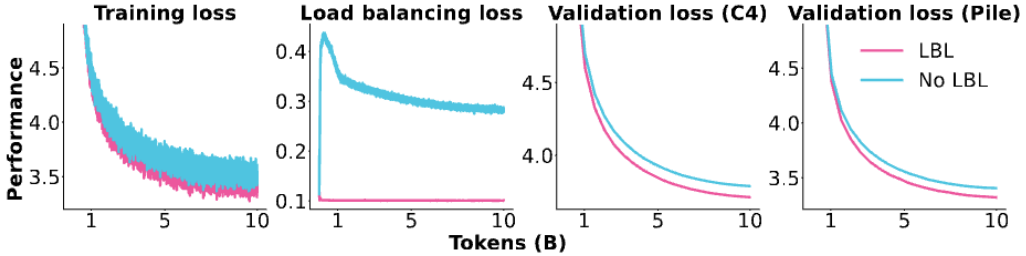



Figure 9: **Impact of applying a load balancing loss (LBL).** The training loss plot excludes the load balancing loss for both models. More results, logs, and configurations: https://wandb.ai/ai2-llm/olmoe/reports/Plot-LBL-vs-No-LBL--Vmlldzo4OTkyNDg4

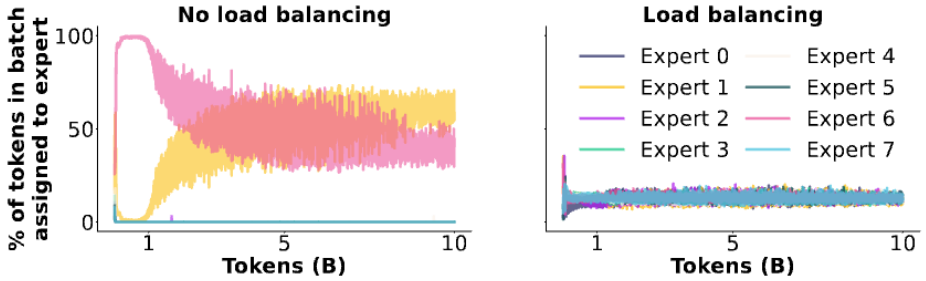



Figure 10: **Expert assignment during training when using or not using a load balancing loss for the first MoE layer.** More results, logs, and configurations: https://wandb.ai/ai2-llm/olmoe/reports/Plot-LBL-vs-No-LBL--Vmlldzo4OTkyNDg4

第 4 讲 · 原页 43–43 · 中文讲解

# 去掉均衡会发生什么：从曲线看到路由塌缩

## 逐图解读

上方依次给出训练损失、负载均衡损失和两个验证损失；训练损失图不把辅助均衡项加进去，才能较公平地看语言建模目标。去掉 LBL 后，即使某段训练损失看似接近，验证曲线仍可能更差。下方两图按专家画实际分配比例：无均衡时部分专家几乎不被使用，另一些长期拥挤；有均衡时各条线更接近均匀负载。曲线噪声并不等于完全随机路由，它反映批次和模型学习的变化。

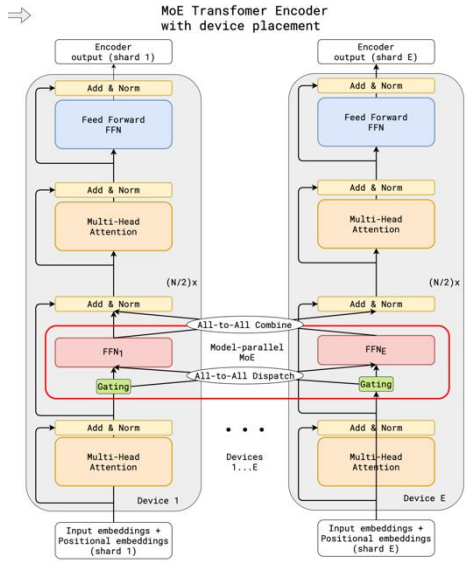
## 编者解释：正反馈与证据边界

早期略受欢迎的专家得到更多 token、更多任务梯度，可能变得更好，再吸引更多 token；冷门专家则越来越难追上。这是路由塌缩的一种直觉机制。删除均衡后性能下降既可能来自参数利用不足，也可能来自溢出或训练不稳；单张图不能精确区分比例。结论是该实验支持保留某种负载控制，并不要求所有模型都采用同一种 auxiliary loss。上一页的 bias 控制和 dropless 实现也必须分别验证。

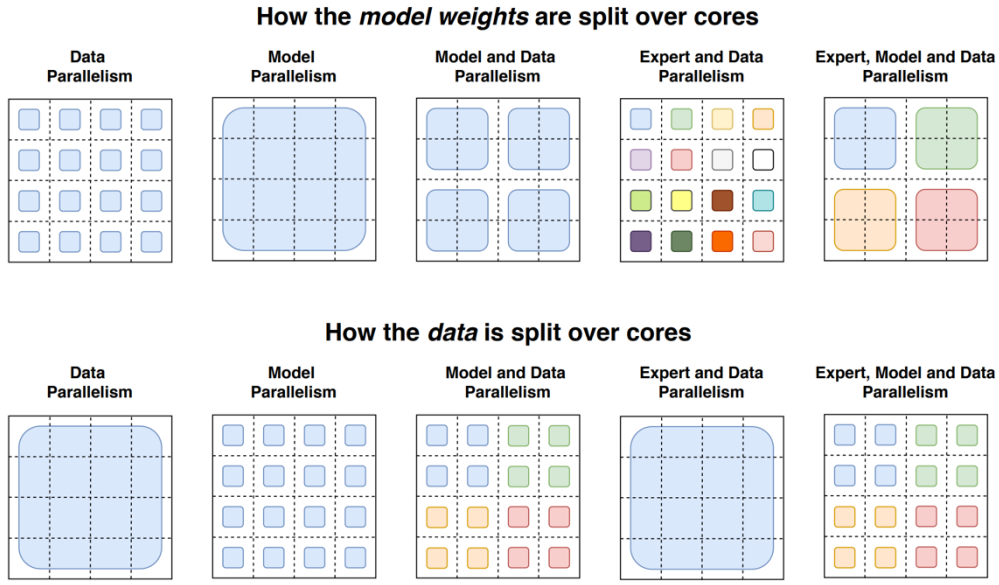
本节参考（完整书目见讲义末尾）：[slides]

# Training MoEs – the systems side

MoEs parallelize nicely – Each FFN can fit in a device

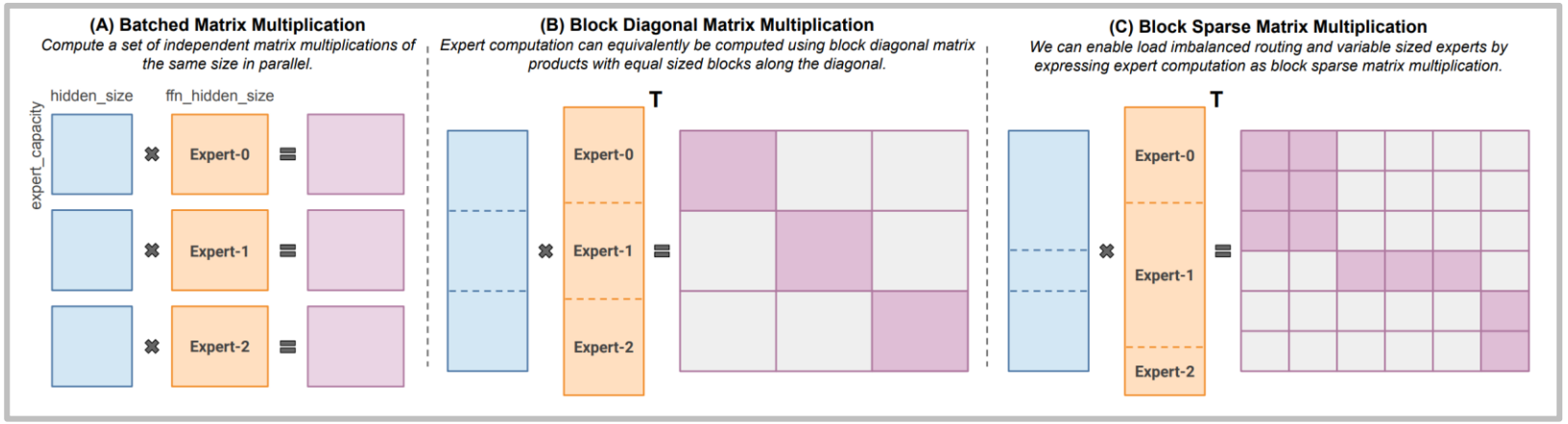



Enables additional kinds of parallelism



# Training MoEs – the systems side

MoE routing allows for parallelism, but also some complexities




Modern libraries like MegaBlocks (used in many open MoEs) use smarter sparse MMs

第 4 讲 · 原页 44–45 · 中文讲解

# 专家并行的真实数据流与稀疏矩阵计算

## 原页 **44**：参数怎么分，数据怎么走

数据并行让各设备处理不同样本并同步梯度；张量/模型并行把同一个算子的权重分片；专家并行把不同 FFN 专家放到不同设备。图上半部展示权重分布，下半部展示数据分布，说明这些方式可以组合。专家路由先对 token 按目标专家排序，发送表示，执行 FFN，再将输出送回原 token 位置并按 gate 合并。一个 token 选多个远端专家时会产生多份分发，反向传播还要反向交换梯度。

## 编者容量例子：**padding** 与 **dropping** 的成本

设 T=8、k=2、E=4，总分配数为 16，均匀目标为每专家 4。若分配数为 (7,5,3,1)，固定每专家容量 4 会丢掉 3+1=4 次专家计算；token 仍可能有残差或另一个专家输出，并不是从序列中删掉整枚 token。若所有专家都 padding 到最大 7，则要安排 28 个位置，只实际用 16 个，浪费 12 个。capacity factor 通常将目标容量放大为约 c·kT/E，降低丢弃风险，但增加空算和缓存。

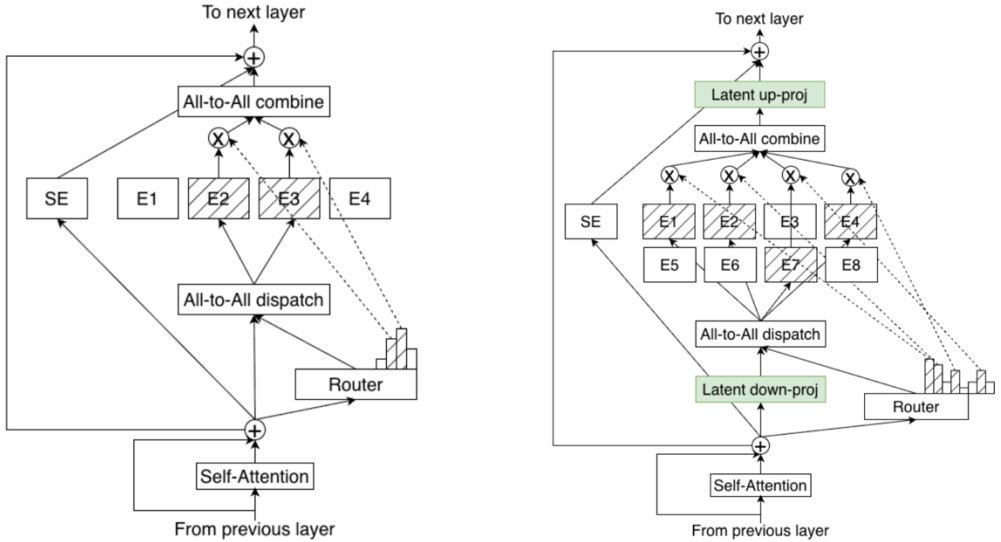
## 原页 **45**：三种矩阵组织

左图把每个专家放入固定尺寸的 batched matrix multiplication，容易产生填充/容量限制；中图将专家权重置于大块对角结构，零区域会造成浪费；右图用块稀疏矩阵表达不同专家收到的不等长 token 集合。MegaBlocks 的思路是用适合加速器的块粒度组织非零计算，支持不丢 token 的工作量。dropless 解决容量截断，不意味着通信消失、负载完全均匀或任何规模都更快，实际仍依赖稀疏内核、块大小与设备布局。

本节参考（完整书目见讲义末尾）：[slides] · [megablocks]

# MoE parallelism and architecture modifications

**2.2. LatentMoE: Hardware-Aware Expert Design for Improved Accuracy per Byte**




(a) Standard MoE architecture.

(b) LatentMoE architecture.

New ideas from Nemotron 3 – down-projecting the activations to reduce communication

第 4 讲 · 原页 46–46 · 中文讲解

# LatentMoE：把通信的向量也变窄

## 沿图追踪一次前向

标准 MoE 图中，router 决定专家之后，完整隐藏向量通过 all-to-all 发出；右图在 dispatch 前先做 latent down-proj，把 d 维表示压到 r 维，专家在较窄空间处理，combine 后再 up-proj 回 d 维。图中共享专家与路由专家的结构需按各自分支看，不能假设所有分支都必须跨网发送。这个设计把模型架构和通信约束放在一起考虑。

## 编者通信估算与代价

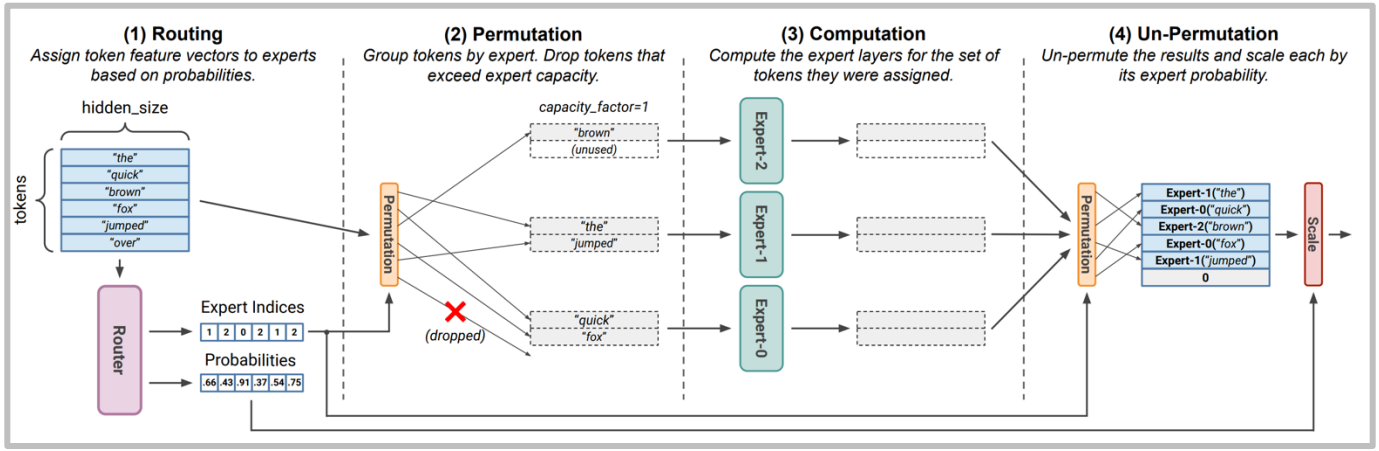
若一个 token 发给 k 个远端专家，每个数占 b 字节，忽略元数据、重复目的设备合并和网络协议，单程传输量约 kd b，压缩后约 kr b。取 d=4096、r=1024、k=4、b=2，分别是 32768 字节与 8192 字节，理论 payload 为四分之一；往返还要计输出。实际加速不会自动等于四倍，因为有投影计算、消息启动、链路共享与其他层时间。r 过小还可能限制表达，因此原页所谓硬件友好是质量、FLOPs、参数和通信量之间的联合取舍，不是无损压缩保证。

本节参考（完整书目见讲义末尾）：[slides] · [nemotron]

# Fun side issue – stochasticity of MoE models

MoEs can have additional stochasticity beyond normal models..

Why would a MoE have additional randomness?




Token dropping from routing happens at a *batch* level – this means that other people's queries can drop your token!

第 4 讲 · 原页 47–47 · 中文讲解

# 批次依赖与额外不确定性

## 为什么相同输入可能走出不同结果

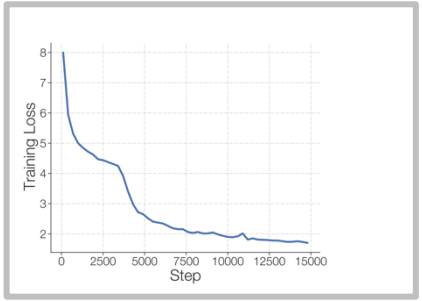
原页图从 routing、permutation、computation 到 un-permutation 画出容量限制造成的丢弃。若某专家最多收 4 个 token，当前请求原本排第 4 个能执行，混入另一请求后排第 5 个就可能被截断。其 MoE 分支输出随共批次请求变化，最终文字也可能变化，即使温度为 0。这里的“不确定性”可以来自确定的批次竞争，不必有随机数生成器参与。

## 限定条件与编者诊断思路

这不是所有 MoE 必然发生的行为：dropless 路由不因容量而丢分支，合理的推理调度也能避免此类影响。还存在与 MoE 无关的浮点归约顺序、内核非确定性和采样噪声。若要定位，应固定权重、输入和解码设置，比较独立请求与不同 batch 搭配，记录专家选择、容量与丢弃计数。不能因为输出变化就证明服务商启用了 token dropping，也不能因为是 MoE 就断言其他用户一定会干扰你的结果。

本节参考（完整书目见讲义末尾）：[slides]

# Issues with MoEs - stability




[7]Exponential functions have the property that a small input perturbation can lead to a large difference in the output. As an example, consider inputting 10 logits to a softmax function with values of 128 and one logit with a value 128.5. A roundoff error of 0.5 in `bfloat16` will alter the softmax output by 36% and incorrectly make all logits equal. The calculation goes from $\frac{\exp(0)}{\exp(0)+10\cdot\exp(-0.5)} \approx 0.142$ to $\frac{\exp(0)}{\exp(0)+10\cdot\exp(0)} \approx 0.091$. This occurs because the max is subtracted from all logits (for numerical stability) in softmax operations and the roundoff error changes the number from 128.5 to 128. This example was in `bfloat16`, but analogous situations occur in `float32` with larger logit values.

[Zoph 2022]

**Solution:** Use Float 32 just for the expert router (sometimes with an aux z-loss)

$$L_z(x) = \frac{1}{B}\sum_{i=1}^{B}\left(\log\sum_{j=1}^{N} e^{x_j^{(i)}}\right)^2 \tag{5}$$

# Z-loss stability for the router

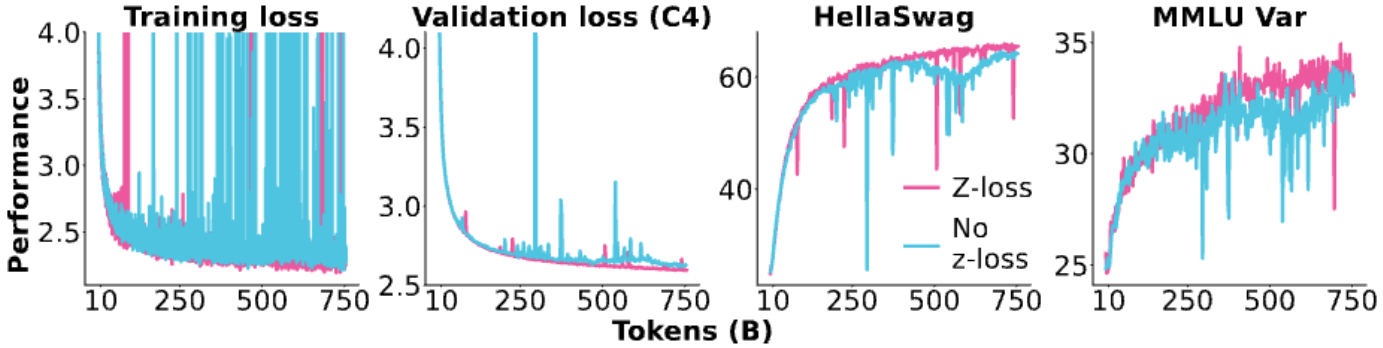



Figure 11: **Router z-loss.** We compare adding router z-loss with a loss weight of 0.001 versus no additional z-loss. More results, logs, and configurations: https://wandb.ai/ai2-llm/olmoe/reports/Plot-Zloss-vs-none--Vmlldzo4NDM4NjUz

What happens when we remove the z-loss?

第 4 讲 · 原页 48–49 · 中文讲解

# Router 的数值稳定性与 z-loss

## 为什么 **router** 值得单独使用 **FP32**

路由决策依赖分数排序；两个 logits 很接近时，舍入足以改变专家集合，而指数/归一化又可能放大数值问题。原页摘录用大 logits 的有限精度例子说明风险。稳定 softmax 先减去最大值可避免显式指数溢出，但不能恢复输入量化时已丢失的差异。FP32 router 是选择性提高精度，其他 FFN 仍可低精度执行，因此额外成本通常比全模型 FP32 小。

## 编者推导：**z-loss** 约束的是 **log-partition**

对 token t 的专家 logits a_{ti}，定义 z_t=log Σ_i exp(a_{ti})，L_z=(1/T)Σ_t z_t²，再乘一个小系数加入总损失。导数 ∂L_z/∂a_{ti}=(2/T)z_t softmax_i(a_t)。实际求 z 用 logsumexp 稳定计算。softmax 对共同平移 a+c 不变，但 z 会变为 z+c；因此 z-loss 对 softmax 忽略的整体尺度/平移提供约束。它不是简单的 Σa_i²，也不是鼓励各专家负载均匀的损失。

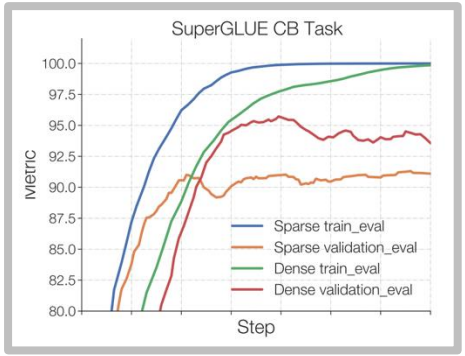
## 编者手算与原页 **49** 曲线

两专家 logits 都为 0，则 z=ln2、L_z≈0.48045（单 token）。把它们都改为 −ln2，softmax 仍为 (1/2,1/2)，但 z=0、L_z=0。反过来，z=0 并不保证概率均匀，例如 logits=(ln0.9,ln0.1) 同样 z=0，却为 (0.9,0.1)。原页 49 去掉 z-loss 的曲线出现更多尖峰与波动，支持该配方的稳定性收益；不能因此把 z-loss 当成所有不稳定问题的充分解法，学习率、初始化、精度和负载也需配合。

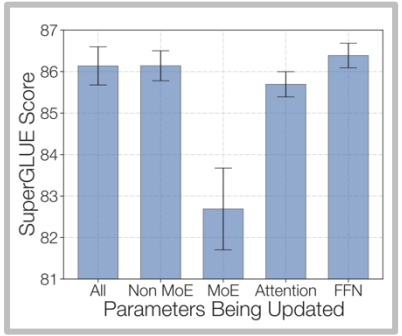
本节参考（完整书目见讲义末尾）：[slides] · [stmoe]

# Issues with MoEs – fine-tuning

Sparse MoEs can overfit on smaller fine-tuning data




Zoph et al solution – finetune non-MoE MLPs




DeepSeek solution – use lots of data 1.4M SFT

**Training Data.** For training the chat model, we conduct supervised fine-tuning (SFT) on our in-house curated data, comprising 1.4M training examples. This dataset spans a broad range of categories including math, code, writing, question answering, reasoning, summarization, and more. The majority of our SFT training data is in English and Chinese, rendering the chat model versatile and applicable in bilingual scenarios.

第 4 讲 · 原页 50–50 · 中文讲解

# 微调风险：更多参数怎样接收更少数据

## 读训练与验证分叉

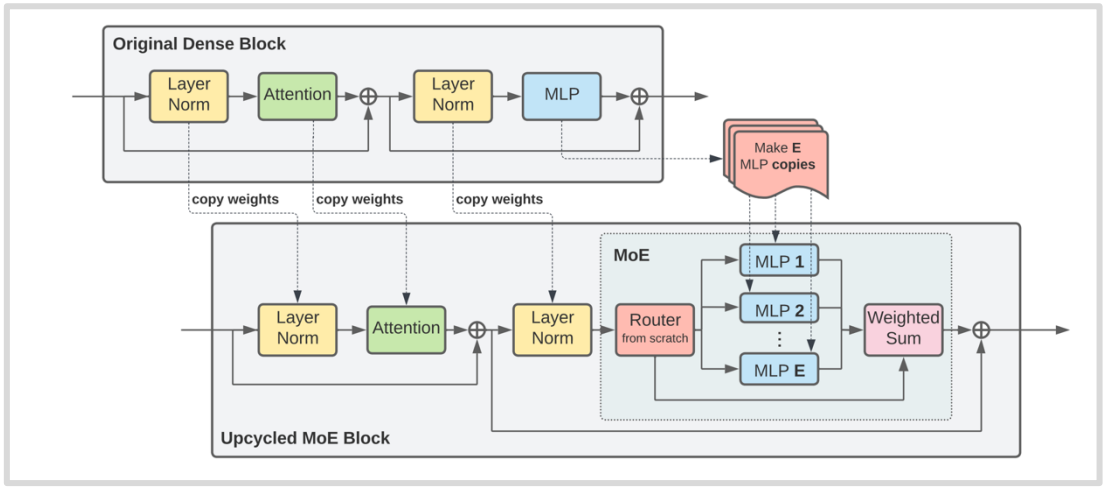
原页上方小数据微调曲线显示训练得分继续变好而验证得分可能下降。MoE 每个专家只接收一部分 token，数据很少时，一些专家有效训练样本更少，路由也可能快速偏向局部模式。总参数大而每个专家的数据稀疏，是理解过拟合风险的一个角度；不能简单说稀疏模型在所有微调任务上一定比稠密模型差。

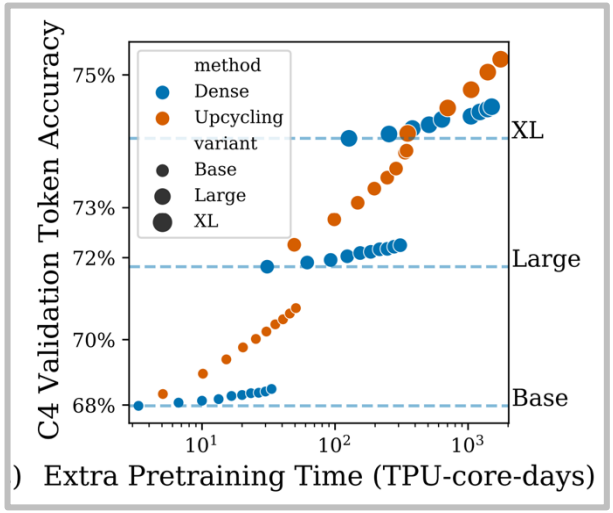
## 两类应对思路与边界

左下实验比较只更新不同参数子集：在所测任务中，限制到非 MoE 等部分可缓解泛化问题；注意区分 non-MoE parameters 与只更新某个普通 FFN 的具体设置，原图标签并不完全相同。右侧 DeepSeek 使用约 1.4M SFT 样本，是以更多且多样的数据支持全模型适配的路线。“冻结专家”与“增加数据”分别改变可学习自由度和有效样本量，不是二选一的普适公式。实际应监控验证集、专家负载及任务分布，不能只根据训练损失下降判断微调成功。

本节参考（完整书目见讲义末尾）：[slides]

# Other training methods - upcycling




Can we use a pre-trained LM to initialize a MoE?

# Upcycling example - MiniCPM

Uses the MiniCPM model (topk=2, 8 experts, ~ 4B active params).

| Model | C-Eval | CMMLU | MMLU | HumanEval | MBPP | GSM8K | MATH | BBH |
|---|---|---|---|---|---|---|---|---|
| Llama2-34B | - | - | 62.6 | 22.6 | 33.0† | 42.2 | 6.24 | **44.1** |
| Deepseek-MoE (16B) | 40.6 | 42.5 | 45.0 | 26.8 | 39.2 | 18.8 | 4.3 | - |
| Mistral-7B | 46.12 | 42.96 | **62.69** | 27.44 | 45.20 | 33.13 | 5.0 | 41.06 |
| Gemma-7B | 42.57 | 44.20 | 60.83 | 38.41 | 50.12 | 47.31 | 6.18 | 39.19 |
| MiniCPM-2.4B | 51.13 | 51.07 | 53.46 | 50.00 | 47.31 | 53.83 | 10.24 | 36.87 |
| **MiniCPM-MoE (13.6B)** | **58.11** | **58.80** | 58.90 | **56.71** | **51.05** | **61.56** | **10.52** | 39.22 |

Table 6: Benchmark results of MiniCPM-MoE. † means evaluation results on the full set of MBPP, instead of the hand-verified set (Austin et al., 2021). The evaluation results of Llama2-34B and Qwen1.5-7B are taken from their technical reports.

Simple MoE, shows gains from the base model with ~ 520B tokens for training

# Upcycling example – Qwen MoE

**Qwen MoE** – Initialized from the Qwen 1.8B model top-k=4, 60 experts w/ 4 shared.

| Model | #Parameters | #(Activated) Parameters | MMLU | GSM8K | HumanEval | Multilingual | MT-Bench |
|---|---|---|---|---|---|---|---|
| Mistral-7B | 7.2 | 7.2 | 64.1 | 47.5 | 27.4 | 40.0 | 7.60 |
| Qwen1.5-7B | 7.7 | 7.7 | 64.6 | 50.9 | 32.3 | - | - |
| Gemma-7B | 8.5 | 7.8 | 61.0 | 62.5 | 36.0 | 45.2 | 7.60 |
| DeepSeekMoE 16B | 16.4 | 2.8 | 45.0 | 18.8 | 26.8 | - | 6.93 |
| Qwen1.5-MoE-A2.7B | 14.3 | 2.7 | 62.5 | 61.5 | 34.2 | 40.8 | 7.17 |

Similar architecture / setup to DeepSeekMoE, but one of the first (confirmed) upcycling successes

第 4 讲 · 原页 51–53 · 中文讲解

# Upcycling：从稠密模型复用已学权重

## 原页 **51** 的初始化过程

左图保留稠密模型的注意力、归一化等组件，把原 FFN 权重复制到多个专家，加入新的 router，再继续训练。其动机是利用已有 dense checkpoint 的训练投入，而不是从随机初始化重新学习所有知识。右图横轴是额外预训练时间，纵轴是验证准确率；比较时既可问“已有 checkpoint 的前提下追加预算花在哪里”，也可问“连同前期稠密训练，总预算是否更划算”，两种成本口径不能混用。

## 编者推导：何时初始化保持原函数

设复制后各专家都是同一个 f(x)，若选中 gate 权重和为 1，则 $\Sigma_i g_i f(x)=f(x)$，MoE 分支在初始化可保持原 FFN 输出。若采用截断后不重归一化的 gate，和为 $c<1$，则变成 $c f(x)$；加入共享专家而不调整也可能改变尺度。因此复制权重不自动保证整块函数完全不变。开始训练后，不同输入路由与梯度让专家分化；如果所有专家始终收到完全相同数据和权重，它们不会仅因复制数量增加就自动产生多样性。

## 原页 **52–53** 的实例与成本

MiniCPM 例子用 8 专家、top-2，并经过大量继续训练；Qwen1.5-MoE 例子从 1.8B 稠密模型出发，采用细粒度和共享专家。表格中的提升证明这些具体转换与训练流程可行，但不能省略继续训练的 token 成本，把它说成复制 checkpoint 后立刻获得新能力。更长预算下从头训练 MoE 可能有不同结果，相关消融需对齐初始投入、后续训练和最终激活算量。upcycling 的核心是复用已有能力，再用路由和训练形成条件分工。

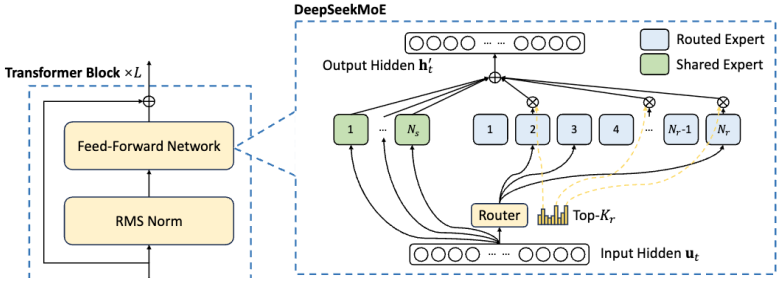
本节参考（完整书目见讲义末尾）：[slides] · [upcycling]

# DeepSeek MoE v1-v2-v3

To wrap up, we'll walk through the DeepSeek MoE architecture.

**V1 (16B – 2.8 active):**

**Shared (2) + Fine-grained (64/4) experts**




**Standard, top-k routing**

$$\mathbf{h}'_t = \mathbf{u}_t + \sum_{i=1}^{N_s} \mathrm{FFN}_i^{(s)}(\mathbf{u}_t) + \sum_{i=1}^{N_r} g_{i,t}\,\mathrm{FFN}_i^{(r)}(\mathbf{u}_t),$$

$$g_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} \in \mathrm{Topk}(\{s_{j,t} | 1 \leqslant j \leqslant N_r\}, K_r), \\ 0, & \text{otherwise}, \end{cases}$$

$$s_{i,t} = \mathrm{Softmax}_i\left(\mathbf{u}_t{}^T \mathbf{e}_i\right),$$

**Standard Aux-loss balancing** (Expert + Device)

$$\mathcal{L}_{\mathrm{ExpBal}} = \alpha_1 \sum_{i=1}^{N_r} f_i P_i,$$

$$f_i = \frac{N_r}{K_r T} \sum_{t=1}^{T} \mathbb{1}(\text{Token } t \text{ selects Expert } i),$$

$$P_i = \frac{1}{T} \sum_{t=1}^{T} s_{i,t},$$

第 4 讲 · 原页 54–54 · 中文讲解

# DeepSeekMoE V1：把前面的部件组装起来

## 一层的计算路径

原页以约 16B 总参数、2.8B 激活参数概括 V1。共享专家对每个 token 执行，路由专家做 top-k 条件执行；细粒度专家减少单个 FFN 的宽度，输出相加后接残差。课件“64/4”应结合前面的粒度表理解为 64 个路由专家、每个相对参考 FFN 缩小至约 1/4，不是 64 除以 4 等于实际专家总数。其模型配置有 2 个共享专家和每 token 6 个激活路由专家。

## 编者串联：结构、路由、训练目标

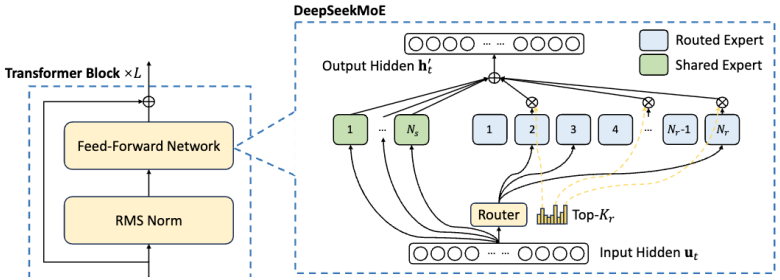
输入 u_t 一方面进入共享 FFN，另一方面用 router 向量得到 softmax affinity；选出路由专家并按 gate 加权。专家级均衡避免少数 FFN 包揽 token，设备级均衡避免某设备成为瓶颈。三种机制分别对应此前的粒度设计、选择函数、训练目标；它们合起来解释为何总参数可以远大于激活参数，但每 token 仍保留共享能力。不要用本页的 16/2.8 比值直接当作理论训练加速倍数，因为通信、稠密组件和数据配方并未被这个比值覆盖。

本节参考（完整书目见讲义末尾）：[slides] · [deepseek-moe]

# DeepSeek MoE v2

## V2 (236B – 21 active):

**Shared (2) + Fine-grained (160/10) experts, 6 active**




**New things:**

Top-M device routing

For DeepSeek-V2, beyond the naive top-K selection of routed experts, we additionally ensure that the target experts of each token will be distributed on at most $M$ devices. To be specific, for each token, we first select $M$ devices that have experts with the highest affinity scores in them. Then, we perform top-K selection among experts on these $M$ devices. In practice, we find that when $M \geqslant 3$, the device-limited routing can achieve a good performance roughly aligned with the unrestricted top-K routing.

Communication balancing loss – balancing both communication in *and* out

$$\mathcal{L}_{\text{CommBal}} = \alpha_3 \sum_{i=1}^{D} f''_i P''_i, \tag{29}$$

$$f''_i = \frac{D}{MT} \sum_{t=1}^{T} \mathbb{1}(\text{Token } t \text{ is sent to Device } i), \tag{30}$$

$$P''_i = \sum_{j \in \mathcal{E}_i} P_j, \tag{31}$$

where $\alpha_3$ is a hyper-parameter called communication balance factor. The device-limited routing mechanism operates on the principle of ensuring that each device transmits at most $MT$ hidden states to other devices. Simultaneously, the communication balance loss is employed to encourage each device to receive around $MT$ hidden states from other devices. The communication balance loss guarantees a balanced exchange of information among devices, promoting efficient communications.

第 4 讲 · 原页 55–55 · 中文讲解

# DeepSeek-V2：从专家负载走向通信负载

## 读架构数字和新增限制

原页 V2 的 236B 总参数与 21B 激活参数继续区分容量与每 token 路径；每层 160 个路由专家、2 个共享专家，每 token 激活 6 个路由专家，专家粒度相对参考 FFN 约 1/10。新增 top-M device routing：先限制一个 token 可发送到的设备集合，再在这些设备上的专家中做选择。M 是设备数限制，k 是专家数，两个数不能互换；同一设备可以有多个选中专家。

## 编者例子：减少通信目的地

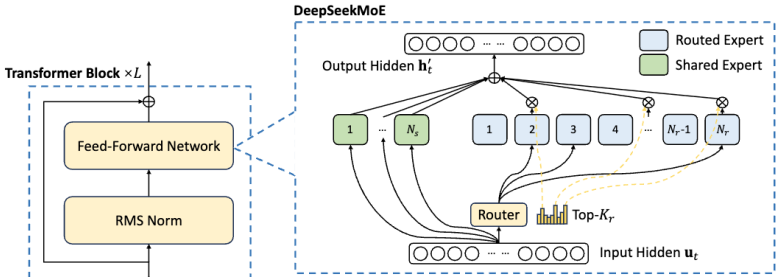
假设 8 个专家分到 4 台设备、每台 2 个，token 选 k=4。无设备限制时可能一台选一个，要联系 4 台；限制 M=2 后只能在两台上的专家中凑齐 4 个，最多联系 2 台。这样可能降低通信开销，但也可能排除全局分数最高的组合，形成质量与网络成本的取舍。页中 communication balancing 进一步关心设备接收/发送负载；计算均衡不充分保证网络均衡，因为相同计算量可能分布在不同数量的来源和目的地。具体 M 与路由实现以模型配方为准。

本节参考（完整书目见讲义末尾）：[slides] · [deepseek2]

# DeepSeek MoE v3

## V2 (671B – 37 active):

**Shared (1) + Fine-grained (258) experts, 8 active**




## New things

Sigmoid+Softmax topK + topM

$$\mathbf{h}'_t = \mathbf{u}_t + \sum_{i=1}^{N_s} \mathrm{FFN}_i^{(s)}(\mathbf{u}_t) + \sum_{i=1}^{N_r} g_{i,t}\,\mathrm{FFN}_i^{(r)}(\mathbf{u}_t),$$

$$g_{i,t} = \frac{g'_{i,t}}{\sum_{j=1}^{N_r} g'_{j,t}},$$

$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} \in \mathrm{Topk}(\{s_{j,t} | 1 \leqslant j \leqslant N_r\}, K_r), \\ 0, & \text{otherwise}, \end{cases}$$

$$s_{i,t} = \mathrm{Sigmoid}\left(\mathbf{u}_t^T \mathbf{e}_i\right),$$

Aux-loss-free + seq-wise aux

$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} + b_i \in \mathrm{Topk}(\{s_{j,t} + b_j | 1 \leqslant j \leqslant N_r\}, K_r), \\ 0, & \text{otherwise}. \end{cases}$$

第 4 讲 · 原页 56–56 · 中文讲解

# DeepSeek-V3：准确读出 sigmoid 路由与均衡

## 原页勘误：保留英文原稿，在此澄清

本页标题为 DeepSeek MoE v3，但下一行写 V2(671B–37 active)，且上方写 258 个细粒度专家。对照 V3 技术报告与原页 35，应该理解为 V3：671B 总参数、每 token 约 37B 激活；MoE 层有 256 个路由专家和 1 个共享专家，每 token 选 8 个路由专家。原页数字和英文不修改，中文标出核对结果，避免把不一致带进实现。

## 按顺序复述一次正确路由

先对每个路由专家算 $a_i=u^Te_i$，再取 $s_i=\text{sigmoid}(a_i)$。在允许的节点/分组范围内，用 $s_i+b_i$ 决定入选集合 I，最终 $g_i=s_i/\Sigma_{j\in I}s_j$（$i\in I$），其他为 0，再把路由专家加权输出与共享专家、残差相加。原页“Sigmoid+Softmax topK”应读为 sigmoid 后的选中分数归一化，而非再做指数 softmax。节点限制压缩通信范围；bias 调整主要专家负载，序列级辅助项仍存在。

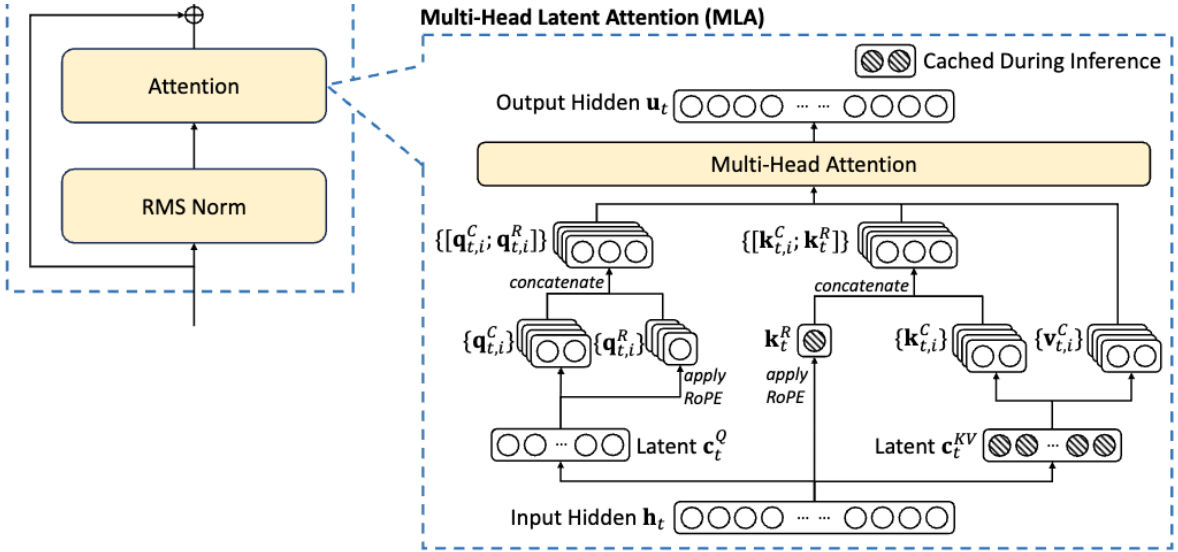
## 编者例子：不该混淆的三种量

若选中两专家的 sigmoid 分数为 0.8、0.4，混合权重就是 2/3、1/3。用于选择的 bias 可以改变谁进入集合，但不能把 $b_i$ 直接当作输出 gate 的一部分。总参数 671B 描述需要存储的全部模型权重，37B 是每 token 的激活路径；256 是每层的路由专家数，8 是每层的选择数，四个数字各自有单位和统计范围。把“8/256”当成全模型激活比例，会忽略共享专家、注意力与其他稠密部分。

本节参考（完整书目见讲义末尾）：[slides] · [deepseek3]

# Bonus: What else do you need to make DeepSeek MoE v3?

**MLA :** Multihead, latent attention

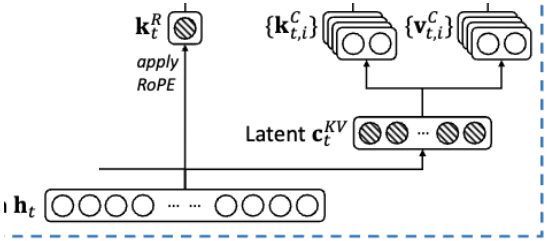



**Basic idea:** express the Q, K, V as functions of a lower-dim, ‘latent’ activation

# What else do you need to make DeepSeek MoE v3?

**Basic idea:** express the Q, K, V as functions of a lower-dim, 'latent' activation

$$\mathbf{c}_t^{KV} = W^{DKV}\mathbf{h}_t,$$
$$\mathbf{k}_t^C = W^{UK}\mathbf{c}_t^{KV},$$
$$\mathbf{v}_t^C = W^{UV}\mathbf{c}_t^{KV},$$




**Benefits:** when KV-caching, we only need to store $c_t^{KV}$, which can be much smaller.
$W^{UK}$ can be merged into the Q projection

(they also compress queries, for memory savings during training)

$$\mathbf{c}_t^Q = W^{DQ}\mathbf{h}_t,$$
$$\mathbf{q}_t^C = W^{UQ}\mathbf{c}_t^Q,$$

**Complexity:** rope conflicts with MLA-style caching.

Without RoPE – $\langle Q, K\rangle = \langle hW^Q, W^{UK}c_t^{KV}\rangle = \langle h\, W^Q W^{UK}, c_t^{KV}\rangle$

With RoPE - $\langle QR_q, R_k K\rangle = \langle hW^Q R_q, R_k W^{UK}c_t^{KV}\rangle = \langle h\, W^Q R_q R_k W^{UK}, c_t^{KV}\rangle$

The solution – Have a few non-latent key dimensions that can be rotated

第 4 讲 · 原页 57–58 · 中文讲解

# MLA：压缩 KV 缓存，以及 RoPE 为什么妨碍吸收

## 编者形状推演：先写清低秩分解

用列向量约定，$h_i \in R^d$，压缩缓存 $c_i=W_D h_i \in R^r$，内容 key 为 $k_i=W_{UK} c_i \in R^{d_k}$，value 为 $v_i=W_{UV} c_i \in R^{d_v}$。对当前 query $q_t$，打分 $q_t^T k_i=q_t^T W_{UK} c_i=(W_{UK}^T q_t)^T c_i$；可以把上投影吸收到当前 query 的计算里。聚合 values 也有 $\Sigma_i \alpha_{ti} W_{UV} c_i=W_{UV}(\Sigma_i \alpha_{ti} c_i)$。于是逐历史位置缓存小 $c_i$，读取与聚合在潜在空间进行，最后恢复输出。多头各有相应投影，但共享潜在缓存的思路不变。

## 编者缓存例子与注意力类型

若把所有头合计的 K、V 宽度分别设为 4096，原始缓存每 token 是 8192 个数；若 latent 宽度 $r=512$，另存 64 维位置 key，则变为 576 个数，约为原来的 7%。这是假设性数量例子，不是本页模型的精确配置；dtype、头数、位置分支等都会影响真实字节数。MLA 仍可以对所有历史位置做 softmax 注意力，压缩的是每位置存储表示，并没有自动把注意力变成线性状态模型。其缓存仍随 token 数增长。

## 编者推导：RoPE 使位置相关矩阵留在中间

若完整 key 和 query 都旋转，令位置 t、i 的旋转矩阵为 $R_t$、$R_i$，则分数为 $(R_t q_t)^T (R_i W_{UK} c_i)=q_t^T R_t^T R_i W_{UK} c_i$。中间 $R_t^T R_i$ 随历史位置 i 变化，一般不能把它与 $W_{UK}$ 合并成一个对所有历史都相同的 query 投影。原页为直觉省略了一些转置，这里用统一列向量约定写清。不能未经条件假设就让旋转矩阵和上投影交换位置。

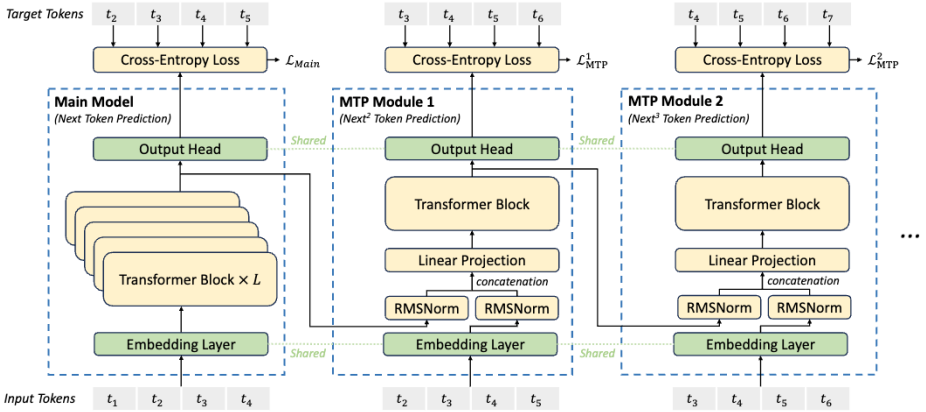
## 解耦位置分支与阅读图示

图中的内容 key 从 latent 恢复，另有较小的 RoPE key/query 分支显式携带位置，打分由内容项加位置项组成。因此缓存需保存潜在内容 $c_i$ 和位置 key，而不是严格只保存 $c_i$。query 的低秩投影还能影响训练激活开销，但 query 通常不是随历史累计保存的 KV cache。MLA、MoE、Mamba 分别压缩每 token 缓存、每 token 激活参数、历史状态，三者名字都涉及效率却不是同一个优化。

本节参考（完整书目见讲义末尾）：[slides] · [deepseek2] · [deepseek3]

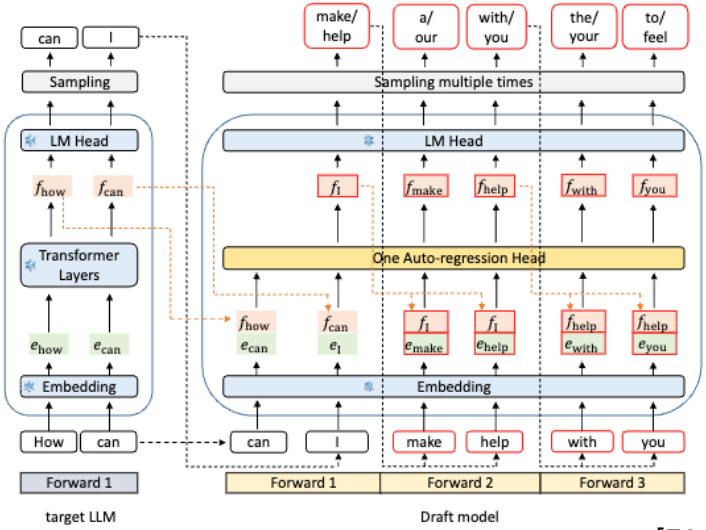
# What else do you need to make DeepSeek MoE v3?

**MTP:** Have small, lightweight models that predict multiple steps ahead




(But they only do MTP with one token ahead)

[Deepseek v3]




[EAGLE]

$$\mathbf{h}_i'^k = M_k[\mathrm{RMSNorm}(\mathbf{h}_i^{k-1}); \mathrm{RMSNorm}(\mathrm{Emb}(t_{i+k}))],$$

$$\mathbf{h}_{1:T-k}^k = \mathrm{TRM}_k(\mathbf{h}_{1:T-k}'^k),$$

$$P_{i+k+1}^k = \mathrm{OutHead}(\mathbf{h}_i^k).$$

(See paper for ablations)

第 4 讲 · 原页 59–59 · 中文讲解

# MTP：额外预测目标与推测解码的区别

## 左图多步目标：未来 **token** 不是白送给主模型

常规语言模型用位置 t 的表示预测 x_{t+1}。MTP 在主干表示上接额外小模块，结合相应后续 token 的 embedding，训练更远一步的目标；教师强制时这些 token 已知。图中每层都有自己的交叉熵，但训练图必须保持预测目标相对于该分支输入的因果顺序，不能把待预测 token 自己输入进去。原页给的串联结构不是同一个 logits 简单复制多次，而是额外模块沿预测深度推进。

## 原稿措辞澄清与推理用途

V3 的 MTP depth=1 表示在通常的下一 token 之外，再预测一个额外 token：主干预测 x_{t+1}，额外模块对应 x_{t+2}。原页“only one token ahead”容易误读为只做普通下一词训练，应理解为只用一层额外预测深度。训练时可以用 MTP 增加学习信号，推理时可以丢弃额外模块，或用它提出草稿 token 交由主模型验证。

## 右图 **EAGLE** 与编者边界

右图用草稿树展示推测解码：廉价模型/模块提出候选，目标模型批量验证，接受符合验证规则的前缀，拒绝处回退。MTP 提供可用于草稿的模块，但不等于无需验证就能并行输出任意多个 token；效果依赖草稿接受率和验证成本。要保持目标采样分布，还需正确的接受/拒绝与校正机制，不能把“挑中主模型最高分”当成通用采样校正证明。课件只是连接这两条路线，本节不扩展成另一个解码算法教程。

本节参考（完整书目见讲义末尾）：[slides] · [deepseek3]

# MoE summary

- MoEs take advantage of sparsity – not all inputs need the full model
  - Discrete routing is hard, but top-k heuristics seem to work
- Lots of empirical evidence now that MoEs work, and are cost-effective

第 4 讲 · 原页 60–60 · 中文讲解

# 把整讲串起来：节省的资源与保留的能力

## 沿两条主线回看原稿

线性注意力先去掉不能重排的 softmax，再用结合律将两两交互改为状态读写；SSM 门控让旧信息可衰减，GDN 让当前键方向可纠错；混合层补充完整注意力的访问能力，DSA 则保留历史存储但选择少量位置精算。MoE 沿另一条轴稀疏化每 token 的 FFN 计算，以总参数增加换取条件容量，同时承担路由、负载与通信复杂度。两条路线可组合，但节省的资源不同。

## 编者验收问题：能否独立解释一张架构图

遇到新架构，先问跨 token 的信息放在哪里：完整 KV、压缩 KV 还是固定状态？再问每个 token 执行哪些参数：所有 FFN 还是 top-k 专家？最后问它的吞吐曲线是在什么硬件、batch、长度和质量条件下测得。若能从这些问题推到状态形状、缓存增长、激活参数、通信路径及训练梯度，就掌握了本讲的工具。末尾自测要求亲手算这些量，而不是记住模型名字与榜单名次。

本节参考（完整书目见讲义末尾）：[slides]

# 自测与参考答案

## 1. 结合律到底改变了什么？为什么不能原样用于 softmax？

编者解答：$QK^T$ 先生成 n×n；$K^TV$ 先生成 d_k×d_v。恒等 ρ 下两者精确相等，成本从 $n^2(d_k+d_v)$ 变成 2nd_kd_v。softmax 非线性地依赖整行打分，不能凭矩阵结合律移过它。去 softmax 是模型改变，之后重排才是精确代数等价。

## 2. 为什么生成时不能对每个位置都用最终 $K^TV$？

最终状态包含未来 token。因果模型第 t 个输出只能使用 $S_t=\Sigma_{i\le t}k_i v_i^T$，其并行等价式为 $(M\odot QK^T)V$，M 是下三角 mask。使用 S_n 会把尚未可见的值混进去；若严格只看 i<t，则应读取 S_{t−1}。

## 3. 两条键值的归一化输出是多少？需要缓存什么？

取 $k_1=(1,0)^T$、$k_2=(1,1)^T$、$v_1=2$、$v_2=4$、$q_2=(1,1)^T$，φ 为恒等。$S_2=(6,4)^T$，$z_2=(2,1)^T$，分子 10、分母 3，输出 10/3。缓存状态矩阵 S 与向量 z；本例特征非负且分母非零。只算分子会得到 10，不是同一归一化注意力。

## 4. 给定连续 SSM，如何求两步状态？

dh/dτ=−h+u，区间长度 Δ=ln2、输入区间内恒定。A_bar=e^(−Δ)=1/2，$B_bar=\int_0^{\Delta}e^{(-s)}ds=1/2$。$h_0=0$，$u_1=2$、$u_2=4$，所以 $h_1=1$、$h_2=2.5$。直接把连续系数 A=−1 当成离散乘数会得到另一个系统；Euler 也只是离散近似。

## 5. 如何合并两步 scan？能交换次序吗？

若 $F_1(S)=a_1S+b_1$、$F_2(S)=a_2S+b_2$，先 1 后 2 得 $a_2a_1S+a_2b_1+b_2$。$a_1=a_2=1/2$、$b_1=1$、$b_2=2$ 时为 S/4+2.5；交换后为 S/4+2。运算满足结合律而不满足交换律，prefix scan 必须保持序列顺序。训练输入已知时可并行，生成下一 token 仍依赖前一步结果。

## 6. GDN 的 β=0 是否完全不更新？什么时候能完全擦除键方向？

β=0 时 $S_t=\gamma S_{t-1}$，没有新写入但仍可能遗忘。只有 γ=1 才完全不变。$I-kk^T$ 在 ‖k‖=1 时才是正交投影；β=1 时移除当前键方向的旧内容，再写入新值。未归一化键或其他 β 不能直接称作完整正交擦除。

## 7. 7:1 混合是否严格线性？DSA 是否无需存历史？

两者都不能这样说。固定比例的全注意力层仍给混合模型贡献 $O(n^2)$ prefill 项；线性层主要降低系数

和缓存需求。DSA 保存可寻址历史 KV 并选择少量位置做主注意力，indexer 若扫全历史也保留轻量二次项，不能只按主注意力的 nk 估计全部成本。

## 8. MoE 总参数与激活参数怎样算？

共享非专家部分 100M、每专家 50M、8 个路由专家选 2 个、无共享专家，得到总参数 100+8×50=500M，激活参数 100+2×50=200M。若增加一个同样大小的始终启用共享专家，则分别为 550M 与 250M。单 token 少执行不等于只需存激活部分，整批可能用到全部专家。

## 9. top-k 前后归一化为什么影响输出？

全体概率 (0.6,0.3,0.1)，top-2，专家输出 10 和 20：保留原概率得 12，选中后重新归一化得 40/3。V3 若 sigmoid affinity 为 (0.8,0.4)，选中后除以和得 (2/3,1/3)，不是 softmax(0.8,0.4)。bias 影响入选，不应直接加进混合 gate。

## 10. 把 8 个大专家拆成 16 个小专家，为什么要同时改变 k？

若每专家宽度减半，为保持激活专家主算量，k 应由 2 增至 4；总宽度保持 8m，激活宽度保持 2m。选择组合从 28 增至 1820，但路由、通信与小矩阵效率会变化。组合数更多只是表达灵活性的线索，必须消融才能判断质量收益。

## 11. 负载损失的梯度为什么不能说成所有概率同时下降？

L=αNΣ_i f_iP_i，固定硬分配时，概率梯度为 αNf_i/T。但 p 来自 softmax，其 logit 梯度为 (αN/T)p_i(f_i−Σ_jp_jf_j)，各维和为零。热门专家相对下降，冷门专家相对上升。N=2、T=4、f=(0.75,0.25)、p=(0.6,0.4) 时 logit 梯度为 (0.06α,−0.06α)。

## 12. 为什么 auxiliary-loss-free 不能译成完全无辅助损失？

V3 用在线 bias 调节专家级主要负载，拥挤者下调、空闲者上调；它仍保留序列级辅助均衡项。因此名称限定的是主要负载控制机制，不能据此从实现中删掉所有辅助项。z-loss 与均衡项也各有不同目标。

## 13. 容量固定与 padding 分别浪费多少？

T=8、k=2、E=4，分配为 (7,5,3,1)。固定容量 4 会丢弃 3+1=4 次专家分支；统一 padding 到 7，则安排 28 个位置，实际仅 16 个，空算 12 个。块稀疏 dropless 可避免按这种固定容量丢弃，但不消除真实负载差异和通信。

## 14. z-loss 为零能推出负载均衡吗？

不能。两个 logits 为 (ln0.9,ln0.1) 时 logsumexp=log(0.9+0.1)=0，z-loss 为零，但路由概率为

(0.9,0.1)，明显不均匀。z-loss 约束数值尺度，均衡损失约束分配，二者不能互相替代。

## 15. 复制稠密 FFN 为多个专家是否保持模型函数？

若各专家都等于 f，且被选权重和为 1，则混合输出仍为 f；若截断 gate 后权重和小于 1，或增加共享专家而不调整尺度，则不一定。初始化时函数可相同，但质量提升依赖后续路由分化和继续训练，不能把复制权重当作免费新能力。

## 16. MLA 为什么需要额外位置分支？

没有 RoPE 时 $q^TW_UKc=(W_UK^Tq)^Tc$，上投影可吸收到当前 query，历史仅存 latent 内容。完整 RoPE 会产生 $q^TR_t^TR_iW_UKc$，随历史 i 变化，通常无法统一吸收。解耦小位置分支后，缓存包括内容 latent 与位置 key；它仍随 n 增长，不是定长状态。

## 17. V3 的 MTP depth=1 预测什么？能直接跳过验证吗？

常规主干预测 x_{t+1}，额外一层 MTP 预测 x_{t+2}，不是只做普通 next-token prediction。推理可将额外模块用于草稿，但接受候选需要目标模型验证；若要保持目标采样分布还需正确的校正机制。MTP 训练信号、草稿模型和推测解码流程是相关但不同的概念。

# 参考资料

**[slides] Stanford CS336 Spring 2026 Lecture 4**，官方原课件；本地原稿为覆盖基准

https://github.com/stanford-cs336/lectures/blob/main/lecture_04.pdf

**[captions]** 第 4 讲公开视频字幕，第三方保存，非教师审校逐字稿

https://jingzechen.github.io/assets/courses/stanford-cs336-s26/lectures/004/captions.source.srt

**[linear] Katharopoulos et al., Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention**，原论文

https://proceedings.mlr.press/v119/katharopoulos20a.html

**[mamba] Gu and Dao, Mamba: Linear-Time Sequence Modeling with Selective State Spaces**，原论文；零阶保持离散化背景

https://arxiv.org/html/2312.00752v2

**[mamba2] Dao and Gu, Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality**，原论文，**SSM** 与扫描背景

https://arxiv.org/html/2405.21060v1

**[gdn] Yang et al., Gated Delta Networks: Improving Mamba2 with Delta Rule**，原论文

https://arxiv.org/html/2412.06464v3

**[switch] Fedus et al., Switch Transformers**，原论文，**top-1** 与负载均衡

https://jmlr.org/papers/v23/21-0998.html

**[deepseek-moe] Dai et al., DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models**，原论文

https://arxiv.org/abs/2401.06066

**[deepseek2] DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model**，原技术报告

https://arxiv.org/abs/2405.04434

**[deepseek3] DeepSeek-V3 Technical Report**，原技术报告；用于路由、专家数和 **MTP** 勘误

https://arxiv.org/html/2412.19437v2

**[olmoe] OLMoE: Open Mixture-of-Experts Language Models**，原论文；课件所示消融的来源

https://arxiv.org/html/2409.02060v2

**[stmoe] Zoph et al., ST-MoE: Designing Stable and Transferable Sparse Expert Models**，原论文

https://arxiv.org/abs/2202.08906

**[megablocks] Gale et al., MegaBlocks: Efficient Sparse Training with Mixture-of-Experts**，原论文

https://arxiv.org/abs/2211.15841

**[upcycling] Komatsuzaki et al., Sparse Upcycling: Training Mixture-of-Experts from Dense Checkpoints**，原论文

https://arxiv.org/abs/2212.05055

**[nemotron] NVIDIA Nemotron 3: Efficient and Open Intelligence**，官方白皮书；**LatentMoE**

https://arxiv.org/abs/2512.20856

**[dsa] DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models**，原技术报告；**DSA**

https://arxiv.org/abs/2512.02556